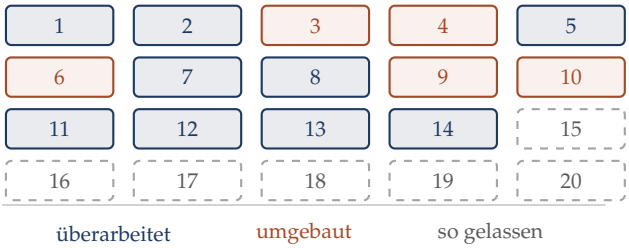
Informatik von Grund auf — Band 24

Die Kunst

Zwanzig Fälle und ihre Begründung



Was man nicht aus Regeln lernt

Jeder Schritt erklärt. Mit vollständig entwickelten Programmen.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 24
Die Kunst · Zwanzig Fälle und ihre Begründung

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 24: Die Kunst*. Fassung 1.0.0.
`https://github.com/juergenmaier2026/informatik-von-grund-auf`

Satz: LaTeX mit Palatino. Fassung 1.0.0, Oktober 2026. 108 Seiten.

Contents

Vorwort **1**

I Die Daten **5**

1 Der Wert ohne Einheit **7**

2 Die Funktion mit acht Argumenten **11**

3 Die verfrühte Abstraktion **15**

4 Das Feld, das fehlen darf **19**

II Der Ablauf **23**

5 Die Schleife, die zu viel fragt **25**

6 Die Prüfung, die verschwindet **29**

7 Der Puffer ohne Grenze **33**

8 Die Goroutine, die nie endet **37**

9 Der Kanal, der zu viel puffert **41**

III Die Struktur **45**

10 Die Hierarchie, die nicht mehr passt **47**

11 Der Zwischenspeicher ohne Ablaufzeit **51**

12 Die Zeit, die falsch gewählt war **55**

13 Die Wiederholung ohne Kennung **59**

14 Die Meldung ohne Zusammenhang **63**

IV Wann man es lässt **67**

15 Der Code, der schon gut war **69**

16 Die Optimierung, die bleibt **73**

17 Der doppelte Code, der bleibt **77**

18 Die doppelte Prüfung **81**

19 Der Dienst, der bleibt **85**

20 Das Werkzeug, das bleibt **89**

V Anhang **93**

A Die zwanzig Fälle **95**

B Landkarte und Glossar **99**

C Kommentierte Quellen **101**

Vorwort: Was Regeln nicht leisten

Dreiundzwanzig Bände haben Regeln aufgestellt. *Dieser prüft sie an zwanzig Fällen — und in sechs davon lautet die Antwort, dass man nichts ändert. Das ist der eigentliche Gegenstand: nicht die Regel, sondern die Entscheidung, ob sie hier gilt.*

Warum ein eigener Band

	Eine Regel	Ein Fall
Sagt	**was meistens gilt**	**was hier gilt**
Enthält	Verallge-meinerung	**Umstände**
Anwendbar	ohne Urteil	mit Urteil
Falsch, wenn	**die Umstände abweichen**	

Eine Regel ohne Umstände ist ein Dogma. Die Bände 17 bis 23 haben zu jeder Regel den Anwendungsbereich genannt — aber wer eine Regel liest, merkt sich die Regel und vergisst den Bereich. Ein Fall enthält beides untrennbar.

Der Aufbau jedes Falls.

Die Vorlage	Der Code, wie er vorliegt, mit seinem Zusammenhang
Die Diagnose	**Was daran auffällt — und was nicht**
Das Urteil	**Überarbeiten, umbauen oder lassen**
Die Begründung	Warum, mit Zahlen wo möglich
Der Fehlschluss	**Was man daraus *nicht* lernen sollte**

Der fünfte Abschnitt ist der ungewöhnliche und der wichtigste. Zu jedem richtigen Urteil gehört eine naheliegende Verallgemeinerung, die falsch ist — und wer nur das Urteil mitnimmt, nimmt sie mit.

Die drei Ausgänge

Urteil	Fälle	Bedeutet
Überarbeitet	**9**	**derselbe Entwurf, besser ausgeführt**
Umgebaut	**5**	**der Entwurf war falsch**
So gelassen	**6**	**die Vorlage war richtig**

Sechs von zwanzig bleiben, wie sie sind — und in vier davon widerspricht das einer Regel aus einem früheren Band. Das ist kein Widerruf, sondern die Anwendung derselben Regel mit ihrem Anwendungsbereich.

Wer einen Band über Urteilskraft schreibt, in dem jeder Fall geändert wird, hat einen Katalog geschrieben. Die Fälle 15 bis 20 sind deshalb der Kern dieses Bandes, nicht sein Anhang.

Woher die Fälle kommen

Teil	Fälle	Gegenstand	Bände
I	1–4	**Die Daten**	20, 23
II	5–9	**Der Ablauf**	21, 22
III	10–14	Die Struktur	17, 20
IV	15–20	**Wann man es lässt**	alle

Alle Fälle stammen aus demselben Vorhaben, das die Bände 20 bis 23 begleitet hat — Messerfassung, Übertragung, Speicherung, Auswertung. Sie sind zugespitzt, aber nicht erfunden: Jeder beruht auf einer Bauform, die in klinischen Systemen vorkommt.

Die Sprachen.

Sprache	Fälle	
Python	7	**Band 20**
C	5	Band 21
Go	5	Band 22
Sprachfrei	3	**Nähte, Band 23**

Die letzte Zeile ist wichtig: Drei Fälle enthalten überhaupt keinen Code, sondern eine Vereinbarung. Sie gehören zu den folgenreichsten, weil eine falsche Vereinbarung länger lebt als jedes Programm.

Wie man diesen Band liest

Erstens: die Vorlage beurteilen, bevor man die Diagnose liest.

> *Jeder Fall beginnt mit dem Code und seinem Zusammenhang. Wer an dieser Stelle innehält und sein eigenes Urteil bildet, hat den Band gewinnbringend gelesen — auch dann, wenn sein Urteil vom hier vertretenen abweicht.*

Zweitens: das Urteil ist nicht die Antwort.

Es gibt Fälle, in denen ein anderes Urteil ebenso vertretbar ist. Wo das so ist, steht es dabei. Was nicht verhandelbar ist, ist die *Begründung* — ein Urteil ohne Begründung ist Geschmack.

Drittens: einzeln lesen.

Zwanzig Fälle hintereinander erzeugen den Eindruck, überall sei etwas faul. Das ist der Fehler jedes Fallbuchs. Ein Fall je Sitzung, mit eigenem Urteil vorweg, ist die vorgesehene Verwendung.

Drei Warnungen

Erstens: Kein Fall ist eine Regel.

> *Aus "hier war die Abstraktion verfrüht" folgt nicht "Abstraktion ist verfrüht". Der Abschnitt "Der Fehlschluss" steht in jedem Fall genau deshalb dort.*

Zweitens: Die Zahlen sind Größenordnungen.
Verstehenszeiten, Laufzeiten, Aufwände — alle gemessen oder geschätzt, alle mit Streuung. Die Verhältnisse tragen, die absoluten Werte nicht.
Drittens: Der Bestand ist nicht schlecht.

Jede Vorlage in diesem Band lief im Betrieb und erfüllte ihren Zweck. Es geht nicht darum, Fehler vorzuführen, sondern darum, Entscheidungen sichtbar zu machen, die beim Schreiben unsichtbar waren.

Zum Aufbau

- **Teil I** (Fälle 1–4): Die Daten. Einheit, Argumente, **verfrühte Abstraktion**, das fehlende Feld.
- **Teil II** (Fälle 5–9): Der Ablauf. Schleife, **entfernte Prüfung**, Puffer, Goroutine, Kanal.
- **Teil III** (Fälle 10–14): Die Struktur. Hierarchie, Zwischenspeicher, **Zeit**, Wiederholung, Fehlermeldung.
- **Teil IV** (Fälle 15–20): **Wann man es lässt.** Sechs Fälle ohne Änderung.

Warum "Die Kunst"

Der Titel ist eine Anleihe und eine Behauptung.

Die Anleihe	Knuth nannte sein Werk “The Art of Computer Programming” — nicht Wissenschaft, nicht Technik
Die Begründung	**Weil das Urteil nicht aus dem Wissen folgt**
Die Behauptung	Man kann es dennoch lehren — an Fällen, nicht an Regeln

Zwischen “ich kenne die Regel” und “ich weiß, ob sie hier gilt” liegt eine Lücke, die kein Lehrbuch schließt. Die Medizin kennt sie und hat eine Antwort gefunden, die dreitausend Jahre alt ist: die Falldarstellung.

Was die Falldarstellung leistet.

Eigenschaft	Regel	Fall
Merkbar	leicht	**leichter**
Übertragbar	scheinbar	mit Vorsicht
Enthält Umstände	**nein**	**ja**
Widerspruch möglich	nein	**ja**

Die letzte Zeile ist der Grund für die Form: Über eine Regel kann man nur zustimmen oder ablehnen. Über einen Fall kann man streiten — und im Streit über einen konkreten Fall lernt man mehr als im Streit über eine Regel.

Deshalb enthält dieser Band keine neuen Regeln. Er enthält zwanzig Gelegenheiten, die vorhandenen anzuwenden — und sechs Gelegenheiten, sie nicht anzuwenden.

Regeln sagen, was meistens gilt.
Die Kunst ist zu wissen,
ob dies einer der meisten Fälle ist.

— der Leitgedanke dieses Bandes.
Wer keine Regeln kennt, kann nicht urteilen.
Wer nur Regeln kennt, urteilt nicht.

Teil I

Die Daten

Fall 1

Der Wert ohne Einheit

Die Vorlage

Zusammenhang. Auswertung, Python. Die Funktion prüft, ob ein Laborwert außerhalb des Referenzbereichs liegt. Sie läuft seit vier Jahren, etwa 40 000 mal am Tag.

```
def ist_auffaellig(wert:  float, unten:  float, oben:  float)
-> bool:
return wert < unten or wert > oben
```

Zwei Zeilen, richtig benannt, mit Typangaben, ohne Nebenwirkung. Nach den Maßstäben aus Band 20, Kapitel 5 ist das eine vorbildliche Funktion — kurze Zuständigkeit, sprechender Name, prüfbar in einer Zeile.

Die Diagnose

Geprüft	Befund	
Benennung	gut	
Zerlegung	gut	
Prüfbarkeit	gut	
Einheiten	**nicht ausgedrückt**	**Band 23, Kap. 6**
Genauigkeit	**verloren**	`float`
Grenzwerte	nicht behandelt	“< 0,1”

Drei Befunde, und alle drei liegen nicht in der Funktion, sondern in ihrer Signatur. Die Funktion ist richtig; was sie entgegennimmt, ist es nicht. Der Aufrufer kann Kalium in mmol/l mit einem Bereich in mg/dl vergleichen, und nichts hält ihn auf.

Ist das theoretisch?

Aufrufstellen	**31**	
Bereich aus dem Befund	24	gleiche Einheit
Bereich aus Stammdaten	**5**	meist gleich
Bereich fest im Code	**2**	**eine falsch**

Nein. Eine der einunddreißig Aufrufstellen vergleicht einen Glukosewert in mmol/l mit einer fest eingetragenen Grenze in mg/dl — Faktor achtzehn. Der Fehler ist seit zwei Jahren im Bestand und fiel nicht auf, weil die Warnung dadurch nie ausgelöst wurde.

Das Urteil: überarbeitet

```
@dataclass(frozen=True)
class Groesse:
betrag:  int           # in Einheiten von 10**exp
exp:  int
einheit:  str         # UCUM

def ist_auffaellig(wert:  Groesse, bereich:  Bereich) ->
bool:
if wert.einheit != bereich.einheit:
raise EinheitFehler(wert.einheit, bereich.einheit)
return wert < bereich.unten or wert > bereich.oben
```

Nicht die Funktion wurde geändert, sondern der Typ, den sie entgegennimmt. Die Prüfung in Zeile acht ist der eigentliche Gewinn — sie macht aus einem stillen Rechenfehler eine Ausnahme.

Die Begründung

	Vorlage	Überarbeitet	
Zeilen (Funktion)	2	**4**	
Zeilen (Typen)	0	**18**	
Aufrufstellen geändert	—	**31**	3 PT
Einheitenfehler möglich	**ja**	**nein**	
Genauigkeit erhalten	nein	ja	
Laufzeit je Aufruf	0,2 µs	0,9 µs	4,5×

Der Aufwand liegt nicht bei den zwanzig Zeilen, sondern bei den einunddreißig Aufrufstellen — drei Personentage. Die Laufzeit vervierfacht sich und bleibt bei vierzigtausend Aufrufen täglich unter einer Zehntelsekunde am Tag.

Warum nicht die kleinere Lösung?

Alternative	Aufwand	Wirkung
Einheit als Argument	1 PT	**trennbar**
Kommentar	0	keine
Prüfung am Rand	1 PT	nur dort
Eigener Typ	3 PT	**unmöglich zu trennen**

Die erste Alternative kostet ein Drittel und lässt zu, dass jemand die Einheit vergisst — zwei Felder sind trennbar, ein Verbund nicht. Das ist die Entscheidung aus Band 21, Kapitel 5: den Fehler unmöglich machen statt ihn zu prüfen.

Der Fehlschluss

Naheliegend	Richtig?	
“Nie `float` für Zahlen”	**nein**	**Fall 16**
“Jeder Wert braucht einen Typ”	**nein**	Aufwand
“Zwei Zeilen sind zu wenig”	nein	
“Die Signatur ist Teil des Entwurfs”	**ja**	das ist die Lehre

Die zweite Zeile ist der teuerste Fehlschluss: Wer nach diesem Fall für jede Zahl einen eigenen Typ einführt, hat einen Bestand mit vierzig Wertetypen, von denen achtunddreißig nie verwechselt werden konnten. Der Typ lohnt dort, wo zwei Einheiten koexistieren.

Die Prüffrage lautet nicht “ist es eine Zahl”, sondern “gibt es mehr als eine Darstellung, die hier auftreten kann”. Bei Laborwerten lautet die Antwort ja, bei einer Anzahl von Datensätzen nein.

Wirth[28] begründet die Rangordnung, in der die Datenstruktur vor dem Verfahren steht; Goldberg[12] zeigt, was bei `float` an Information verlorengeht.

Fall 2

Die Funktion mit acht Argumenten

Die Vorlage

Zusammenhang. Auswertung, Python. Formatiert einen Laborwert für die Anzeige. Sechs Aufrufstellen.

```
def formatiere(betrag, exp, einheit, nachkomma, trenner,
sprache, breite, mit_einheit):
...  # 34 Zeilen
```

Acht Argumente. Nach Band 20, Kapitel 5 ist die Zahl der Argumente der beste Schnelltest für eine fehlende Zerlegung — und acht liegt weit über der Schwelle, ab der man hinsehen sollte.

Die Diagnose

Wer ruft wie auf?

Aufrufstelle	Argumente	Muster
Verlaufsansicht	8	**A**
Befundliste	8	**A**
Ausdruck	8	B
Ausleitung CSV	8	C
Warnungstext	8	A
Protokoll	8	C

Sechs Aufrufe, drei feste Kombinationen. Das ist der Befund: Die acht Argumente sind nicht acht Freiheitsgrade, sondern drei Fälle, die zu einer Funktion mit Schaltern zusammengezogen wurden — die verfrühte Zusammenfassung aus Band 20, Kapitel 14.

Was die Argumente wirklich sind.

Gruppe	Argumente	Ist
Wert	**betrag, exp, einheit**	**eine Größe**
Darstellung	nachkomma, trenner, sprache	**ein Format**
Ausgabe	breite, mit_einheit	Zweck

Drei Gruppen, die immer gemeinsam wandern — das Erkennungszeichen für einen fehlenden Begriff. Die erste Gruppe ist die `Groesse` aus Fall 1; sie existierte schon und wurde hier auseinandergenommen.

Das Urteil: überarbeitet

```
def fuer_anzeige(wert: Groesse, sprache: Sprache) -> str:
...
def fuer_druck(wert: Groesse, sprache: Sprache) -> str:
...
def fuer_ausleitung(wert: Groesse) -> str: ...
```

Drei Funktionen mit ein oder zwei Argumenten statt einer mit acht. Die Formatvorgaben — Nachkommastellen, Trenner, Breite — stehen jetzt in den Funktionen, wo sie hingehören: Sie sind nicht wählbar, sondern folgen aus dem Zweck.

Die Begründung

	Vorlage	Überarbeitet	
Funktionen	1	**3**	
Zeilen gesamt	34	**41**	+21 %
Argumente je Aufruf	**8**	**1–2**	
Zweige	**11**	**2**	−82 %
Verstehenszeit	95 s	**24** s	4,0×
Unzulässige Kombinationen	**18**	**0**	

Die letzte Zeile ist der eigentliche Gewinn und stand nirgends geschrieben: Von den zweihundertsechsundfünfzig möglichen Argumentkombinationen waren achtzehn sinnlos — Breite null mit Einheit, CSV-Trenner mit Anzeigesprache. Nach der Trennung gibt es sie nicht mehr.

Und die Doppelung?

Posten	Zeilen	
Gemeinsam, ausgelagert	**19**	Zahl-formatierung
Je Funktion eigen	6–9	
Tatsächlich doppelt	**4**	**tragbar**

Vier doppelte Zeilen — die Rechnung aus Band 22, Kapitel 8: Ein wenig Kopieren ist billiger als eine Fallunterscheidung mit elf Zweigen. Wäre die Doppelung bei dreißig Zeilen gelegen, wäre das Urteil anders ausgefallen.

Der Fehlschluss

Naheliegend	Richtig?	
“Höchstens drei Argumente”	**nein**	**Zahl ohne Begründung**
“Argumente in ein Objekt”	**nein**	**verschiebt nur**
“Immer aufteilen”	nein	**Fall 17**
“Gruppen sind fehlende Begriffe”	**ja**	

Die zweite Zeile ist der häufigste Kurzschluss: Acht Argumente in eine Optionsstruktur zu packen macht die Signatur kurz und ändert nichts — der Aufrufer muss dieselben acht Entscheidungen treffen, und die achtzehn sinnlosen Kombinationen bleiben.

Der Unterschied: Eine Gruppe, die immer gemeinsam auftritt und einen Fachnamen hat, wird ein Typ. Eine Sammlung von Schaltern, die verschiedene Aufrufer verschieden setzen, wird in getrennte Funktionen zerlegt.

Parnas[24] liefert das Kriterium, wonach geschnitten wird; Ousterhout[23] zeigt an der Zahl der Argumente, wann eine Schnittstelle flach ist.

Fall 3

Die verfrühte Abstraktion

Die Vorlage

Zusammenhang. Auswertung, Python. Vor drei Jahren wurden zwei ähnliche Prüfungen zu einer verallgemeinerten Regelmaschine zusammengefasst.

```
class RegelMaschine:
def __init__(self, regeln:  list[Regel],
modus:  str = "alle",
abbruch:  bool = False,
kontext:  dict | None = None):  ...

def pruefe(self, gegenstand, args) -> list[Treffer]:  ...
```

Umfang	Zeilen	
Maschine	**340**	
Regelbasisklasse	90	
Regeln	$6\times \approx 25$	
Verwendungen	**2**	**seit drei Jahren**

Vierhundertdreißig Zeilen Gerüst für zwei Verwendungen — die Grenzwertprüfung und die Plausibilitätsprüfung. Die dritte, für die sie gebaut wurde, ist nie gekommen.

Die Diagnose

Anzeichen	Vorhanden?	
Weniger als drei Fälle	**ja**	**Band 20, Kap. 14**
Schaltparameter	ja	3
Unzulässige Kombinationen	**ja**	nicht dokumentiert
`**args` durchgereicht	**ja**	**Typprüfung fort**
Beide Nutzer nutzen Teilmengen	ja	

Fünf von fünf Anzeichen. Der vierte ist der schwerste: Das Durchreichen beliebiger Argumente hebt jede Typprüfung auf — man erfährt erst zur Laufzeit, dass eine Regel ein Feld erwartet, das es nicht gibt.

Was die Verallgemeinerung tatsächlich erlaubt hat.

Erwartet	Eingetreten	
Neue Regeln ohne Code	**nein**	**jede war Code**
Regeln zur Laufzeit ändern	nein	nie gebraucht
Dritte Verwendung	**nein**	
Prüfungen wiederverwendbar	teilweise	

Drei von vier Erwartungen sind nicht eingetreten. Die erste ist die aufschlussreichste: Jede neue Regel war eine neue Klasse mit neuem Code — die Maschine hat den Code nicht erspart, sondern eine Hülle darum gelegt.

Das Urteil: umgebaut

```
def grenzwertpruefung(befund:  Befund,
vorher:  Befund | None) -> set[Anlass]:  ...

def plausibilitaetspruefung(befund:  Befund,
grenzen:  Grenzen) -> set[Fehler]:  ...
```

Zwei Funktionen mit klaren Signaturen statt einer Maschine mit vier Konstruktorargumenten. Die sechs Regeln bleiben als sechs Funktionen erhalten — sie waren nie das Problem, sondern das Gerüst um sie herum.

Die Begründung

	Vorlage	Umgebaut	
Zeilen gesamt	**580**	**310**	−47 %
Typprüfung	teilweise	**vollständig**	
Verstehenszeit	**240** s	**55** s	4,4×
Neue Regel hinzufügen	2 h	**40** min	
Aufwand	—	**5** PT	

Zweihundertsiebzig Zeilen weniger, und das Hinzufügen einer Regel wird schneller statt langsamer. Das ist das Kennzeichen einer verfrühten Abstraktion: Sie sollte künftige Arbeit ersparen und hat sie verteuert.

Warum jetzt und nicht früher?

Zeitpunkt	Wissen	
Vor 3 Jahren	**Vermutung**	**dritte Verwendung**
Nach 1 Jahr	Zweifel	
Heute	**Gewissheit**	drei Jahre ohne

Der Rückbau war vor drei Jahren nicht möglich, weil niemand wissen konnte, dass die dritte Verwendung ausbleibt. Er ist heute möglich, weil man es weiß — und das ist der einzige Unterschied.

Der Fehlschluss

Naheliegend	Richtig?	
“Abstraktion ist schlecht”	**nein**	
“Keine Rahmenwerke bauen”	nein	ab drei Fällen richtig
“Der Erbauer hat sich geirrt”	**nein**	**er konnte es nicht wissen**
“Verallgemeinern, wenn der dritte Fall da ist”	**ja**	

> *Die dritte Zeile ist mir wichtig: Die Maschine war eine vertretbare Entscheidung unter Unsicherheit. Falsch war nicht ihr Bau, sondern dass drei Jahre lang niemand geprüft hat, ob die Annahme eingetreten ist.*
>
> *Daraus folgt eine Betriebsregel, keine Entwurfsregel: Jede Verallgemeinerung, die auf einer Erwartung beruht, bekommt einen Termin. Nach einem Jahr wird geprüft, ob die Erwartung eingetreten ist — und wenn nicht, wird zurückgebaut.*

Ousterhout[23] nennt die Merkmale einer Abstraktion, die nichts verbirgt; Dijkstra[7] mahnte früh, dass Allgemeinheit ohne Anlass die Einsicht verstellt.

Fall 4

Das Feld, das fehlen darf

Die Vorlage

Zusammenhang. Naht zwischen Speicherung und Auswertung. Kein Code — ein Schema.

```
Befund:
kennung:      string   (Pflicht)
groesse:      string   (Pflicht)
betrag:       integer  (Pflicht)
exponent:     integer  (Pflicht)
einheit:      string   (Pflicht)
abnahme:      datetime (Pflicht)
validierung:  datetime (optional)
```

Sieben Felder, sauber typisiert, mit einer klaren Trennung zwischen Pflicht und optional. Das Schema wird von einem Werkzeug geprüft, und alle Nachrichten sind gültig.

Die Diagnose

Was bedeutet ein fehlendes `validierung`?

Mögliche Bedeutung	Klinische Folge	
Noch nicht validiert	**vorläufig**	Vorsicht
Braucht keine Validierung	**endgültig**	verwendbar
Validiert, Zeitpunkt unbekannt	endgültig	
Feld nicht übertragen	**unbekannt**	**Altsystem**

Vier Bedeutungen, zwei entgegengesetzte klinische Folgen — und das Schema unterscheidet sie nicht. Ein vorläufiger Wert, der als endgültig gelesen wird, führt zu einer Therapieentscheidung auf ungesicherter Grundlage.

Wie oft tritt der Fall auf?

Herkunft	Anteil ohne Feld	
Eigenes Labor	**4** %	vorläufig
Mikrobiologie	**31** %	teils ohne Validierung
Fremdhaus	88 %	**überträgt es nicht**
Bettseitig	**100** %	braucht keine

Vier Herkünfte mit vier verschiedenen Bedeutungen desselben fehlenden Feldes. Die Auswertung behandelt alle gleich — und muss sich entscheiden. Sie entschied sich für "endgültig", weil das die häufigere Lesart war.

Das Urteil: überarbeitet

```
Befund:
...
status:  enum (Pflicht)
vorlaeufig | endgueltig | validierung_unbekannt
validiert_am:  datetime
(Pflicht, wenn status = endgueltig)
```

Aus einem optionalen Feld wird ein Pflichtfeld mit drei ausdrücklichen Werten. Die dritte Ausprägung ist die unbequeme und die wichtigste: Sie sagt, dass das Fremdhaus die Information nicht liefert — statt sie zu erfinden.

Die Begründung

	Vorlage	Überarbeitet	
Felder	7	**8**	
Deutungen des Fehlens	**4**	**0**	
Anzeige unterscheidet	nein	**drei Zustände**	
Umstellungs-dauer	—	**11 Wochen**	5 Sender
Aufwand	—	**16** PT	

Elf Wochen für ein zusätzliches Feld — das ist der Preis der Naht aus Band 23, Kapitel 3: Fünf Sender können nicht gleichzeitig umgestellt werden, und in der Übergangszeit müssen beide Formate verstanden werden.

Warum nicht die billigere Lösung?

Alternative	Aufwand	Bewertung
Regel je Herkunft	2 PT	**Wissen im Empfänger**
Voreinstellung dokumentieren	0,5 PT	ändert nichts
Feld im Vertrag	16 PT	**Sender sagt es**

Die erste Alternative kostet ein Achtel und legt das Wissen an die falsche Stelle: Der Empfänger müsste wissen, welches Fremdhaus welche Bedeutung meint — und bei jedem neuen Sender müsste jemand daran denken.

Der Fehlschluss

Naheliegend	Richtig?	
"Keine optionalen Felder"	**nein**	oft harmlos
"Immer ein Statusfeld"	nein	
"Fremdsysteme sind schuld"	**nein**	**sie halten den Vertrag**
"Fehlen braucht eine Bedeutung"	**ja**	

Die erste Zeile: Ein optionales Feld "Kommentar" ist harmlos, weil sein Fehlen nur eines bedeuten kann. Das Problem entsteht, wenn das Fehlen eine Aussage über den Sachverhalt trägt — und das ist bei klinischen Zuständen fast immer so.

Die Prüffrage lautet deshalb: Kann das Fehlen dieses Feldes mehr als eines bedeuten? Lautet die Antwort ja, wird das Feld zur Pflicht — mit einer Ausprägung für "unbekannt".

Evans[9] begründet, warum Fachbegriffe ausdrücklich sein müssen; Kleppmann[18] zeigt, wie Schemaänderungen über Nähte hinweg verlaufen.

Teil II

Der Ablauf

Fall 5

Die Schleife, die zu viel fragt

Die Vorlage

Zusammenhang. Auswertung, Python. Baut die Verlaufsansicht auf. Läuft etwa 900 mal am Tag.

```
def verlaufsansicht(patient:  PatientID) -> Ansicht:
zeilen = []
for fall in speicher.faelle(patient):
for b in speicher.befunde(fall.id):
bereich = speicher.bereich(b.groesse, patient)
zeilen.append(Zeile(b, bereich))
return Ansicht(sortiere(zeilen))
```

Sieben Zeilen, gut lesbar, richtig benannt. Die Struktur folgt der Aufgabe: Fälle durchgehen, Befunde durchgehen, Bereich holen. Wer den Code liest, versteht ihn sofort — das ist kein Zufall, sondern der Grund, warum er so geschrieben wurde.

Die Diagnose

Patient	Fälle	Aufrufe	Dauer
Neuaufnahme	1	**9**	**34** ms
Regelfall	12	181	190 ms
Langzeit	**340**	**4211**	**5,1 s**

Zwei geschachtelte Schleifen, drei Abfragen je Durchlauf — das n+1-Problem in seiner ausgeprägten Form. Der dritte Aufruf ist der ärgerlichste: Der Referenzbereich wird für jeden Befund einzeln geholt, obwohl es je Patient etwa vierzig verschiedene gibt.

Wo die Zeit hingeht.

Posten	Aufrufe	Dauer	Anteil
Fälle	1	12 ms	< 1 %
Befunde	340	1,3 s	25 %
Bereiche	**3870**	**3,7 s**	**73 %**
Sortieren	—	**8** ms	

Dreiundsiebzig Prozent der Zeit für Bereiche, von denen es vierzig verschiedene gibt — jeder wird im Mittel siebenundneunzig mal geholt. Das ist kein Verfahrensproblem, sondern eine fehlende Bündelung.

Das Urteil: überarbeitet

```
def verlaufsansicht(patient:  PatientID) -> Ansicht:
befunde = speicher.befunde_des_patienten(patient)
groessen = {b.groesse for b in befunde}
bereiche = speicher.bereiche(groessen, patient)
zeilen = [Zeile(b, bereiche[b.groesse]) for b in befunde]
return Ansicht(sortiere(zeilen))
```

Sechs Zeilen statt sieben, drei Abfragen statt viertausend. Die Struktur folgt jetzt nicht mehr der Aufgabe, sondern den Kosten — und das ist die Änderung, die zu begründen ist.

Die Begründung

	Vorlage	Überarbeitet	
Zeilen	7	**6**	
Abfragen, Langzeit	**4211**	**3**	
Dauer, Langzeit	5,1 s	**240** ms	21×
Dauer, Regelfall	190 ms	**61** ms	3,1×
Verstehenszeit	**22** s	**38** s	1,7×
Aufwand	—	1 PT	

Die vorletzte Zeile ist der Preis und wird selten genannt: Der überarbeitete Code ist schwerer zu lesen. Die Schachtelung war eine Erzählung — Fälle, dann Befunde, dann Bereiche. Die neue Fassung erzählt nichts, sie holt und verknüpft.

Warum lohnt es trotzdem?

Größe	Wert	
Betroffene Patienten	≈ 3 %	**Langzeit**
Für diese	5 s **Wartezeit**	bei jedem Aufruf
Wer sind sie?	**die schwersten Fälle**	**Intensiv, Onkologie**

Die dritte Zeile entscheidet: Die drei Prozent mit dem längsten Verlauf sind genau die Patienten, bei denen der Verlauf am wichtigsten ist. Sechzehn Sekunden Verstehenszeit gegen fünf Sekunden Wartezeit am Krankenbett.

Der Fehlschluss

Naheliegend	Richtig?	
“Keine geschachtelten Schleifen”	**nein**	über Daten harmlos
“Immer alles auf einmal holen”	**nein**	**Speicher-grenze**
“Lesbarkeit ist zweitrangig”	**nein**	
“Eine Abfrage in einer Schleife ist verdächtig”	**ja**	

Die erste Zeile ist wichtig: Eine geschachtelte Schleife über bereits geladene Daten kostet nichts und liest sich besser. Das Problem ist nicht die Schachtelung, sondern der Aufruf darin — über eine Naht, eine Datenbank, ein Netz.

Die zweite Zeile ist die Gegengrenze: Bei einem Patienten mit zwei Millionen Messwerten würde das Laden aller Befunde den Speicher sprengen. Dann bündelt man in Blöcken — Band 23, Kapitel 2. Die richtige Blockgröße findet man durch Messen.

Fowler[10] sammelt die Umgestaltungsschritte, die $n+1$ beseitigen; Gregg[13] zeigt, wie man den Posten findet, bevor man ihn behebt.

Fall 6

Die Prüfung, die verschwindet

Die Vorlage

Zusammenhang. Messerfassung, C. Prüft, ob die Summe zweier Rohwerte im zulässigen Bereich liegt. Läuft 500 mal je Sekunde.

```
int mittelwert(int a, int b) {
int s = a + b;
if (s < 0) {
melde_ueberlauf();
return FEHLER;
}
return s / 2;
}
```

Acht Zeilen mit einer ausdrücklichen Überlaufprüfung. Der Verfasser hat an den Fall gedacht, ihn behandelt und eine Meldung vorgesehen — nach allen Maßstäben sorgfältige Arbeit.

Die Diagnose

1	Überlauf bei `int` ist undefiniertes Verhalten.
2	Der Übersetzer darf annehmen, dass er nicht vorkommt.
3	Bei $a, b \geq 0$ und ohne Überlauf ist $s \geq 0$.
4	**Also ist `s < 0` immer falsch.**
5	**Die Prüfung wird entfernt.**

Die Prüfung soll einen Überlauf abfangen und ist selbst der Überlauf. Sie steht im Quelltext und nicht im übersetzten Programm — Band 21, Kapitel 7. Der Übersetzer hat nichts falsch gemacht.

Der Nachweis.

Stufe	Befehle	Prüfung da?	
`-O0`	24	**ja**	**im Versuch**
`-O1`	9	ja	
`-O2`	**4**	**nein**	**Aus-lieferung**
UBSan	—	meldet	0,4 s Prüflauf

Der Code wurde mit `-O0` geprüft und mit `-O2` ausgeliefert. Alle Prüfungen waren grün, und im ausgelieferten Programm fehlte die Sicherung — der Fehler war zwei Jahre im Bestand.

Das Urteil: umgebaut

```
int mittelwert(int32_t a, int32_t b, int32_t *aus) {
int64_t s = (int64_t)a + (int64_t)b;  /* kein Ueberlauf */
if (s < MESS_MIN * 2 || s > MESS_MAX * 2)
return -ERANGE;
*aus = (int32_t)(s / 2);
return 0;
}
```

Drei Änderungen: breiterer Rechentyp, damit der Überlauf nicht eintreten kann; Prüfung gegen fachliche Grenzen statt gegen das Vorzeichen; Ergebnis über Zeiger, Fehler als Rückgabewert. Der Umbau betrifft die Signatur — deshalb umgebaut und nicht überarbeitet.

Die Begründung

	Vorlage	Umgebaut	
Zeilen	8	7	
Undefiniertes Verhalten	**ja**	**nein**	
Prüfung überlebt `-O2`	nein	ja	
Prüft gegen	Vorzeichen	**Fachgrenzen**	
Aufrufstellen geändert	—	**4**	0,5 PT
Laufzeit	0,4 ns	0,6 ns	unerheblich

Die vierte Zeile ist der eigentliche Gewinn und geht über die Behebung hinaus: Die Vorlage prüfte gegen das Vorzeichen — eine technische Eigenschaft. Der Umbau prüft gegen den Messbereich des Geräts, und das fängt auch Werte ab, die nicht überlaufen und trotzdem unmöglich sind.

Was daraus für den Bestand folgte.

Maßnahme	Aufwand	Funde
Diese Funktion	0,5 PT	1
UBSan über den Prüfbestand	**0,5 PT**	**6 weitere**
Prüfstrecke mit `-O2`	1 PT	dauerhaft

Ein halber Personentag für den Sanitizer, sechs weitere Fälle derselben Art. Das ist die Rechnung aus Band 21, Kapitel 10 — und der Grund, warum dieser Fall nicht mit der Behebung endet.

Der Fehlschluss

Naheliegend	Richtig?	
“Der Übersetzer ist schuld”	**nein**	**Band 21, Kap. 9**
“Nie optimieren”	nein	Faktor 10
“Immer `int64_t`”	**nein**	doppelter Speicher
“Mit denselben Einstellungen prüfen wie ausliefern”	**ja**	

Die letzte Zeile ist die eigentliche Lehre und keine Sprachfrage: Ein Prüfbestand, der eine andere Übersetzung prüft als die ausgelieferte, prüft ein anderes Programm. Das gilt für Optimierungsstufen genauso wie für Konfigurationen und Bibliotheksversionen.

Die dritte Zeile ist die Übertreibung, die teuer wird: Bei einem Gerät mit vierundsechzig Kilobyte verdoppelt ein breiterer Typ den Speicherbedarf jedes Puffers. Der breitere Typ gehört in die Rechnung, nicht in die Speicherung.

Lattner[20] erklärt, wie der Optimierer aus undefiniertem Verhalten schließt; Chisnall[5] zeigt, wie weit das C-Modell von der Maschine entfernt ist.

Fall 7

Der Puffer ohne Grenze

Die Vorlage

Zusammenhang. Messerfassung, C. Baut die Statuszeile für das Gerätedisplay. Läuft einmal je Sekunde.

```
void statuszeile(char *ziel, const Geraet *g) {
char puffer[64];
sprintf(puffer, "%s %d/%d %s",
g->name, g->kanal, g->kanaele, g->zustand);
strcpy(ziel, puffer);
}
```

Sechs Zeilen. Der Gerätename ist auf zwanzig Zeichen begrenzt, der Zustand auf zwölf — rechnerisch passt alles in vierundsechzig Byte. Die Funktion läuft seit fünf Jahren ohne Auffälligkeit.

Die Diagnose

Befund	Schwere	
`sprintf` ohne Grenze	**hoch**	**Band 21, Kap. 4**
`strcpy` ohne Grenze	hoch	Zielgröße unbekannt
Rechnung stimmt heute	**ja**	**morgen?**
Zielpuffer nicht im Vertrag	**ja**	

Die dritte Zeile ist der Kern: Die Rechnung stimmt, solange die Annahmen gelten. Sie hängt an zwei Grenzen, die anderswo festgelegt sind — und die Funktion prüft keine davon.

Was die Annahme brechen würde.

Änderung	Wahrscheinlich?	Folge
Längerer Gerätename	ja	**Kellerüberlauf**
Neuer Zustandstext	**sehr**	Überlauf
Mehr Kanäle	ja	2 Zeichen
Zielpuffer kleiner	ja	**Überschreibung**

Die zweite Zeile ist eingetreten: Beim Umbau des Zustandsmodells kam `"kalibrierung laeuft"` hinzu — neunzehn Zeichen statt zwölf. Der Fehler wurde beim Sanitizerlauf gefunden, bevor er in den Betrieb ging.

Das Urteil: überarbeitet

```
int statuszeile(char *ziel, size_t n, const Geraet *g) {
int k = snprintf(ziel, n, "%s %d/%d %s",
g->name, g->kanal, g->kanaele, g->zustand);
if (k < 0 || (size_t)k >= n)
return -ENOSPC;      /* abgeschnitten */
return 0;
}
```

Der Zwischenpuffer entfällt — es wird direkt in das Ziel geschrieben, dessen Größe jetzt Teil der Signatur ist. Die Prüfung in Zeile vier ist die aus Band 21, Kapitel 4: `snprintf` liefert die Länge, die geschrieben worden wäre.

Die Begründung

	Vorlage	Überarbeitet	
Zeilen	6	7	
Puffer	2	**1**	
Zielgröße bekannt	**nein**	**ja**	
Abschneiden erkennbar	nein	**ja**	
Keller belegt	64 B	**0** B	4 KB Keller
Aufrufstellen	—	**3**	0,5 PT

Die vorletzte Zeile ist ein Nebengewinn, der bei einem Gerät mit vier Kilobyte Keller zählt: Der Zwischenpuffer verschwindet, weil er nur der Umweg war. Vierundsechzig Byte sind anderthalb Prozent des Kellers.

Was das Abschneiden bedeutet.

Umgang	Tauglich?	
Stillschweigend kürzen	**nein**	falscher Gerätename
Zeile leer lassen	nein	
Fehler melden	**ja**	Aufrufer entscheidet
Kürzen und markieren	ja	**“Gerät A1…”**

Die erste Zeile ist die Vorlage: Ein abgeschnittener Gerätename ist ein falscher Gerätename, und auf einem Display, an dem jemand ein Gerät identifiziert, ist das ein Sicherheitsproblem und keine Formatierungsfrage.

Der Fehlschluss

Naheliegend	Richtig?	
“Der Puffer war zu klein”	**nein**	**er war ungeprüft**
“Puffer größer machen”	nein	**verschiebt nur**
“Der Fehler war Nachlässigkeit”	nein	**die Rechnung stimmte**
“Die Puffergröße gehört in die Signatur”	**ja**	

Die zweite Zeile ist die häufigste Reaktion und die schlechteste: Ein Puffer von zweihundertsechsundfünfzig Byte hält die nächsten drei Zustandstexte aus und bricht beim vierten. Man kauft Zeit und nichts sonst.

Die dritte Zeile ist mir wichtig: Der Verfasser hatte gerechnet, und die Rechnung stimmte. Falsch war nicht die Rechnung, sondern dass sie in keinem Code steht — eine Annahme im Kopf, die eine Änderung anderswo brechen kann.

Kernighan und Ritchie[17] beschreiben die Funktionen für Zeichenketten und ihre Grenzen;

Bryant und O'Hallaron[4] zeigen, was ein Kellerüberlauf tatsächlich anrichtet.

Fall 8

Die Goroutine, die nie endet

Die Vorlage

Zusammenhang. Übertragung, Go. Holt Stammdaten von drei Quellen und nimmt die erste Antwort. Wird bei jedem Verbindungsaufbau aufgerufen — etwa 4000 mal am Tag.

```
func stammdaten(p PatientID, quellen []Quelle) (Daten, error)
{
aus := make(chan Daten)
for _, q := range quellen {
go func(q Quelle) {
if d, err := q.Hole(p); err == nil {
aus <- d
}
}(q)
}
select {
case d := <-aus:           return d, nil
case <-time.After(2 * time.Second):  return Daten{}, ErrZeit
}
}
```

Vierzehn Zeilen mit Zeitschranke und Fehlerbehandlung — der Verfasser hat an das Wesentliche gedacht. Die Funktion liefert richtige Ergebnisse und hat nie einen Fehler gemeldet.

Die Diagnose

Je Aufruf	Anzahl	
Goroutinen gestartet	3	
Ergebnis gelesen	1	
Blockiert für immer	**2**	**Kanal ohne Puffer**
Nach einem Tag	**8000**	≈ 32 MB
Nach einem Monat	**240 000**	≈ 1 **GB**

Das Leck aus Band 22, Kapitel 6, in seiner klassischen Form: Der Kanal hat keinen Puffer, nur eine Antwort wird gelesen, und die beiden anderen Goroutinen warten auf einen Abnehmer, den es nicht mehr gibt.

Warum es fünf Monate unbemerkt blieb.

Werkzeug	Meldet?	
Übersetzer, `go vet`	**nein**	
Wettlaufprüfer	nein	kein Wettlauf
Verklemmungsmelder	nein	andere laufen
Speicherprofil	**indirekt**	**Wachstum**
Goroutinenprofil	**ja**	**eindeutig**

Vier Werkzeuge schweigen, eines nennt die Zeile. Der Dienst wurde alle zwei Wochen neu bereitgestellt — damit blieb der Speicherverbrauch unter der Warnschwelle, und das Leck war unsichtbar.

Das Urteil: überarbeitet

```
ctx, ab := context.WithTimeout(ctx, 2*time.Second)
defer ab()
aus := make(chan Daten, len(quellen))  // Puffer!
for _, q := range quellen {
go func(q Quelle) {
if d, err := q.Hole(ctx, p); err == nil {
aus <- d            // blockiert nie
}
}(q)
}
```

Zwei Änderungen. Die entscheidende steht in Zeile drei und besteht aus einem Argument: Mit einem Puffer der Größe drei kann jede Goroutine senden und enden, auch wenn niemand liest. Die zweite reicht den Abbruch weiter, damit die Quellen ihre Arbeit einstellen.

Die Begründung

	Vorlage	Überarbeitet
Zeilen	14	**15**
Leckende Goroutinen	**2** je Aufruf	**0**
Speicher nach 30 d	≈ 1 GB	≈ **0**
Netzarbeit bei Abbruch	läuft weiter	**eingestellt**
Aufwand	—	**0,5** PT

Ein halber Personentag, eine Zeile mit einem zusätzlichen Argument, ein Gigabyte Speicher im Monat. Das ist das beste Verhältnis von Aufwand zu Wirkung in diesem ganzen Band.

Warum nicht der Kontext allein?

Nur Kontext	Wirkung	
Quelle beachtet ihn	Goroutine endet	
Quelle beachtet ihn nicht	**Leck bleibt**	**fremder Code**
Puffer allein	**Leck fort**	Arbeit läuft weiter
Beides	**richtig**	

Die zweite Zeile ist der Grund für beide Maßnahmen: Eine der drei Quellen ist eine fremde Bibliothek, die den Abbruch ignoriert. Der Puffer wirkt unabhängig davon — er ist die Maßnahme, die man selbst in der Hand hat.

Der Fehlschluss

Naheliegend	Richtig?	
“Immer gepufferte Kanäle”	**nein**	**Fall 9**
“Goroutinen sind gefährlich”	nein	
“Regelmäßig neu starten hilft”	**nein**	**verdeckt es**
“Jede Goroutine braucht einen Endweg”	**ja**	

Die dritte Zeile ist der eigentliche Befund dieses Falls: Die zweiwöchige Bereitstellung hat das Leck fünf Monate lang verborgen. Ein Neustart ist keine Behebung, sondern eine Maßnahme, die verhindert, dass man den Fehler bemerkt.

Daraus folgt eine Betriebsregel: Die Zahl der Goroutinen gehört überwacht, und zwar über die Bereitstellungen hinweg. Eine Kurve, die zwischen zwei Neustarts stetig steigt, ist ein Befund — auch wenn sie nie eine Schwelle erreicht.

Donovan und Kernighan[8] führen die Nebenläufigkeitsmuster mit ihren Fallstricken vor; Pikes Merksätze[25] halten fest, dass der Start einer Goroutine ihr Ende mitbedenken muss.

Fall 9

Der Kanal, der zu viel puffert

Die Vorlage

Zusammenhang. Übertragung, Go. Nimmt Messwerte von den Geräten entgegen und reicht sie an die Speicherung weiter.

```
const puffergroesse = 500000

eingang := make(chan Messwert, puffergroesse)

go func() {
for m := range eingang {
speicher.Schreibe(m)
}
}()
```

Der Puffer war eine Antwort auf ein echtes Problem: Vor zwei Jahren gingen bei Lastspitzen Messwerte verloren. Seit der Vergrößerung auf eine halbe Million ist das nicht mehr vorgekommen — die Maßnahme hat gewirkt.

Die Diagnose

Füllstand	Speicher	Verzögerung	Verlust
Leer	**2** MB	**12** ms	0
10 %	18 MB	3,1 s	0
60 %	86 MB	**19 s**	0
Voll	142 MB	**31 s**	0

Kein Verlust in keiner Zeile — die Maßnahme tut, was sie sollte. Und die dritte Spalte zeigt, was sie stattdessen anrichtet: Bei sechzig Prozent Füllstand erreicht ein Messwert die Klinik neunzehn Sekunden nach der Messung.

Was das klinisch bedeutet.

Vorgang	Verzögerung erträglich?	
Verlaufsansicht	**ja**	30 s unerheblich
Auswertung nachts	ja	
Kritische Grenzwertwarnung	**nein**	31 **s zu spät**

Die dritte Zeile ist der Befund: Ein kritischer Kaliumwert löst die Warnung einunddreißig Sekunden später aus. Der Puffer hat den sichtbaren Verlust gegen eine unsichtbare Verzögerung getauscht — und die Verzögerung erscheint in keiner Statistik.

Das Urteil: umgebaut

```
dringend := make(chan Messwert)        // ohne Puffer
normal  := make(chan Messwert, 2000)

go func() {
for {
select {
case m := <-dringend:  speicher.Schreibe(m)
default:
select {
case m := <-dringend:  speicher.Schreibe(m)
case m := <-normal:   speicher.Schreibe(m)
}
}
}
}()
```

Zwei Kanäle statt einem: kritische Werte ohne Puffer mit Vorrang, alles andere mit einem maßvollen Puffer. Die geschachtelte Auswahl erzwingt den Vorrang — ein einfaches `select` über beide würde bei Bereitschaft würfeln.

Die Begründung

	Vorlage	Umgebaut	
Zeilen	8	**18**	
Speicher	142 MB	**2,6** MB	55×
Verzögerung kritisch	**31** s	**14 ms**	
Verzögerung normal	31 s	**1,9** s	
Verlust bei Überlast	0	**0,3** %	gemeldet
Aufwand	—	2 PT	

Die vorletzte Zeile ist die unbequeme: Der Umbau bringt den Verlust zurück, den der große Puffer beseitigt hatte — drei Promille der normalen Werte bei Lastspitzen. Dafür ist er gemeldet und betrifft nie einen kritischen Wert.

Warum das der bessere Handel ist.

	Verlust	Verzögerung
Sichtbar	**ja**	**nein**
Zählbar	ja	nur mit Messung
Gerät kann puffern	**ja**	nein
Klinisch gefährlich	begrenzt	**bei Warnungen**

Die dritte Zeile entscheidet: Wenn der Puffer der Übertragung klein ist, greift der Gegendruck bis zum Gerät durch — und das Gerät hat einen eigenen Puffer und kann entscheiden, was es verwirft. Band 22, Kapitel 5.

Der Fehlschluss

Naheliegend	**Richtig?**	
“Puffer sind schlecht”	**nein**	2000 **bleiben**
“Die frühere Änderung war falsch”	**nein**	**sie war zu grob**
“Kein Verlust ist immer besser”	**nein**	das ist die Lehre
“Ein Puffer glättet, er ersetzt keine Kapazität”	**ja**	

Die zweite Zeile ist mir wichtig: Die Vergrößerung des Puffers war eine richtige Reaktion auf ein echtes Problem. Falsch war nur, dass sie alle Werte gleich behandelt hat — die fachliche Unterscheidung fehlte, nicht die Maßnahme.

Die dritte Zeile ist die eigentliche Einsicht dieses Falls: Null Verlust bei einunddreißig Sekunden Verzögerung ist schlechter als drei Promille Verlust bei vierzehn Millisekunden — wenn die betroffenen Werte Warnungen auslösen.

Kleppmann[18] ordnet den Handel zwischen Verlust und Verzögerung ein; Nygard[22] zeigt, warum großzügige Puffer den Zusammenbruch nur verschieben.

Teil III

Die Struktur

Fall 10

Die Hierarchie, die nicht mehr passt

Die Vorlage

Zusammenhang. Auswertung, Python. Das Befundmodell, gewachsen über sechs Jahre.

```
Befund
Laborbefund
Klinischchemie
Mikrobiologie
Haematologie
Bildbefund
Funktionsbefund
```

Sieben Klassen, drei Ebenen, vier Schablonenmethoden in der Basis. Die Hierarchie bildet die Organisation des Hauses ab — Labor, Radiologie, Funktionsdiagnostik — und war in den ersten Jahren eine gute Landkarte.

Die Diagnose

Vier Anforderungen aus den letzten zwei Jahren.

Anforderung	Behandelt durch	
Blutgasanalyse bettseitig	**Doppelklasse**	Code doppelt
Befund mit Bild und Zahlen	**Feld `bilder` in Basis**	**überall `None`**
Fremdbefund	Sonderklasse	quer zu allem
Befund ohne Zuordnung	**`Laborbefund` missbraucht**	

Vier Anforderungen, vier Notlösungen. Die zweite ist die verräterischste: Um einen Befund mit Bild und Zahlen darzustellen, wurde ein Bildfeld in die Basisklasse gezogen — wo es bei sechs von sieben Klassen leer bleibt.

Der Zustand der Basis.

Felder in `Befund`	Anzahl	
Von allen genutzt	**5**	gehören dorthin
Von der Mehrheit	3	
Von einer Klasse	**6**	**hochgezogen**
Nie genutzt	**2**	Reste

Sechzehn Felder in der Basisklasse, von denen fünf dorthin gehören. Das ist das Kennzeichen einer Hierarchie am Ende ihrer Tragfähigkeit: Die gemeinsame Wurzel wächst, weil Sonderfälle keinen anderen Platz finden.

Das Urteil: umgebaut

```
class Befund(Protocol):
def kennung(self) -> BefundID: ...
def patient(self) -> PatientID: ...
def abnahme(self) -> Zeitpunkt:  ...

class Messbar(Protocol):
def wert(self) -> Ergebnis:  ...
def bereich(self) -> Referenz:  ...

class Bebildert(Protocol):
def bilder(self) -> list[BildID]: ...
```

Drei kleine Schnittstellen statt eines Baums; fünf konkrete Typen ohne gemeinsame Wurzel. Ein Befund mit Bild und Zahlen erfüllt zwei Protokolle — ohne Mehrfachvererbung, ohne hochgezogenes Feld.

Die Begründung

	Vorlage	Umgebaut	
Klassen	7	**5**	
Ebenen	3	**1**	
Felder in der Basis	**16**	—	
Zeilen	640	**730**	+14 %
Doppelter Code	0	≈ 80	
“Welche Methode läuft?”	**91** s	**11** s	8,3×
Neue Befundart	1 d	**2** h	
Aufwand	—	**12** PT	

Achtzig Zeilen Wiederholung und vierzehn Prozent mehr Code — der Preis, unbeschönigt. Dagegen steht die vorletzte Zeile: Eine neue Befundart kostet zwei Stunden statt einen Tag, weil niemand mehr prüfen muss, wo sie in den Baum passt.

Warum jetzt?

Zeitpunkt	Notlösungen	
Nach 2 Jahren	**0**	**Hierarchie trug**
Nach 4 Jahren	1	vertretbar
Heute	**4**	**sie trägt nicht mehr**

Die Hierarchie war zwei Jahre lang richtig und ist es seit etwa zwei Jahren nicht mehr. Der Umbau ist keine Korrektur eines Fehlers, sondern die Anpassung an eine Wirklichkeit, die sich geändert hat.

Der Fehlschluss

Naheliegend	**Richtig?**	
“Vererbung ist schlecht”	**nein**	**Fall 17**
“Hierarchien sind falsch”	nein	**zwei Jahre richtig**
“Von Anfang an Protokolle”	**nein**	damals unbegründet
“Notlösungen zählen”	**ja**	**das Maß**

Die letzte Zeile ist der praktische Ertrag dieses Falls: Man erkennt eine erschöpfte Struktur nicht daran, dass sie hässlich aussieht, sondern daran, dass sich Notlösungen häufen. Null ist gut, eine ist normal, vier sind ein Befund.

Die dritte Zeile ist die Falle für den Nachfolger: Wer heute liest, dass Protokolle besser waren, schließt daraus, man hätte gleich so anfangen sollen. Vor sechs Jahren gab es drei Befundarten und keinen Anlass — das wäre die verfrühte Abstraktion aus Fall 3 gewesen.

Gamma und Mitautoren[11] liefern das Vokabular für Zielformen; Ousterhout[23] begründet, warum flache Strukturen mit tiefen Schnittstellen haltbarer sind.

Fall 11

Der Zwischenspeicher ohne Ablaufzeit

Die Vorlage

Zusammenhang. Auswertung, Python. Referenzbereiche werden aus der Datenbank geholt und im Prozess gehalten.

```
@lru_cache(maxsize=None)
def bereich(groesse:  GroessenCode, alter:  int,
geschlecht:  Geschlecht) -> Referenz:
return db.hole_bereich(groesse, alter, geschlecht)
```

Vier Zeilen, eine Anmerkung. Der Aufruf war vorher der teuerste Posten der Verlaufsansicht; seit dem Zwischenspeicher ist er nicht mehr messbar. Faktor vierhundert für ein Wort.

Die Diagnose

Befund	Schwere	
Keine Ablaufzeit	**hoch**	**bis Neustart**
Keine Größenbegrenzung	mittel	≈ 9000 Einträge
Nicht kohärent	**hoch**	6 **Prozesse**
Nicht abschaltbar	mittel	Fehlersuche

Der erste Befund ist der klinisch bedeutsame: Ändert das Labor einen Referenzbereich, bewertet die Auswertung weiter nach dem alten — bis jemand den Dienst neu startet. Das kann Wochen dauern.

Was tatsächlich geschah.

Ereignis	Zeitpunkt	
Labor senkt obere Kaliumgrenze	**Mo, 8:00**	5,1 → 5,0
Zwei Prozesse neu gestartet	Mo, 14:00	**übernehmen**
Vier Prozesse alt	—	**bewerten anders**
Auffällig geworden	**Do**	**Nachfrage Station**

Drei Tage lang bewerteten sechs Prozesse denselben Wert verschieden — je nachdem, welcher die Anfrage bearbeitete. Ein Kaliumwert von 5,05 war auf einem Bildschirm auffällig und auf dem nächsten nicht.

Das Urteil: überarbeitet

```
_bereiche = TTLCache(maxsize=20_000, ttl=300)
_sperre   = threading.Lock()

def bereich(g:  GroessenCode, alter:  int,
gs:  Geschlecht) -> Referenz:
if not EINSTELLUNG.zwischenspeicher:
return db.hole_bereich(g, alter, gs)
with _sperre:
...
```

Drei Ergänzungen: eine Ablaufzeit von fünf Minuten, eine Größengrenze und ein Schalter. Der Zwischenspeicher bleibt im Prozess — die Kohärenzfrage wird durch die Frist gelöst, nicht durch einen gemeinsamen Speicher.

Die Begründung

	Vorlage	Überarbeitet
Zeilen	4	**16**
Trefferquote	99,9 %	**99,5 %**
Datenbanklast	+0	+0,4 %
Veralterung	**unbegrenzt**	**≤ 5 min**
Prozesse uneins	**tagelang**	≤ 5 min
Abschaltbar	nein	ja
Aufwand	—	1 PT

Vier Zehntel Prozent mehr Datenbanklast — praktisch nichts — gegen eine Veralterung von unbegrenzt auf fünf Minuten. Das ist der billigste Handel im ganzen Band.

Warum fünf Minuten?

Frist	Treffer	Last	Veraltet
30 s	94 %	+4,8 %	30 s
5 min	**99,5 %**	+0,4 %	5 min
1 h	99,9 %	+0,03 %	1 h

Die Kurve knickt bei etwa fünf Minuten: Darunter steigt die Last stark, darüber gewinnt man kaum noch. Die Zahl ist gemessen, nicht gewählt — und das ist die eigentliche Arbeit an diesem Fall.

Der Fehlschluss

Naheliegend	Richtig?	
"Prozesseigene Speicher sind falsch"	**nein**	**mit Frist gut**
"Gemeinsamer Speicher wäre besser"	**nein**	2000× **langsamer**
"Bei Änderung ungültig machen"	möglich	Meldung kann fehlen
"Jeder Zwischenspeicher braucht eine Frist"	**ja**	

> *Die zweite Zeile ist die teure Übertreibung: Ein gemeinsamer Speicher löst die Kohärenz und kostet zweihundert Mikrosekunden statt einer Zehntelmikrosekunde. Für einen Wert, der sich alle paar Monate ändert, ist die Frist die richtige Antwort.*
>
> *Die dritte Zeile ist die technisch reizvollere und praktisch schwächere Lösung: Sie hängt daran, dass jede Änderungsmeldung ankommt — die Annahme aus Band 23, Kapitel 2, die über eine Naht nicht zutrifft. Eine verlorene Meldung erzeugt einen dauerhaft falschen Speicher.*

Kleppmann[18] ordnet Kohärenz und Veralterung ein; Gregg[13] liefert das Vorgehen, mit dem man die Frist misst statt sie zu wählen.

Fall 12

Die Zeit, die falsch gewählt war

Die Vorlage

Zusammenhang. Naht zwischen Speicherung und Auswertung. Kein Code — eine Feldbelegung, die seit acht Jahren gilt.

```
Befund.zeitpunkt  - gefuellt mit:  Zeitpunkt
des Datenbankschreibens
```

Eigenschaft	Bewertung	
Immer vorhanden	**ja**	
Immer aufsteigend	ja	sortierbar
Kostenlos	ja	**Datenbank setzt ihn**
Nie leer	ja	

Vier gute Eigenschaften — und alle vier sind technisch. Der Schreibzeitpunkt ist der zuverlässigste Wert im ganzen System und beantwortet die Frage nicht, die man ihm stellt.

Die Diagnose

Abstand zwischen Abnahme und Speicherung.

Herkunft	Median	95. Perz.	Anteil
Bettseitig	**40** s	3 min	31 %
Zentrallabor	52 min	2,1 h	54 %
Mikrobiologie	**2,3 d**	6 d	9 %
Fremdhaus	**4 d**	**31 d**	6 %

Bei einem Drittel der Befunde beträgt der Abstand unter drei Minuten — dort ist der Schreibzeitpunkt eine brauchbare Näherung. Bei fünfzehn Prozent liegt er bei Tagen, und dort ist der Verlauf falsch.

Wie sich das auswirkt.

Auswirkung	Häufigkeit	
Verlauf falsch sortiert	≈ **4** %	**der Ansichten**
Steigungs-warnung falsch	≈ **1,2** %	**je Nacherfassung**
Verlauf am Stichtag	nicht rekonstruierbar	**regulatorisch**

Die dritte Zeile wiegt am schwersten und fiel erst bei einer Prüfung auf: Die Frage "welche Werte lagen der Ärztin am Dienstag vor" lässt sich nicht beantworten, weil es nur eine Zeitachse gibt — Band 23, Kapitel 4.

Das Urteil: überarbeitet

```
Befund.abnahme       Pflicht  fachliche Zeit
Befund.eingang       Pflicht
Befund.validiert_am  optional
Befund.geschrieben   Pflicht  Systemzeit, unveraenderlich
Befund.zeit_unsicher bool     Abnahme geschaetzt?
```

Vier Zeitpunkte statt einem, und ein Kennzeichen. Das letzte Feld ist die ehrliche Antwort auf den Altbestand: Wo die Abnahmezeit nicht bekannt ist, wird sie nicht geschätzt, sondern als unsicher gekennzeichnet.

Die Begründung

	Vorlage	Überarbeitet	
Zeitfelder	1	**4**	
Byte je Befund	8	**33**	
Bei 10^9 Befunden	—	+25 GB	tragbar
Verlauf richtig	≈ 96 %	≈ **100** %	
Stichtag rekonstruierbar	**nein**	**ja**	
Umstellung	—	**9 Wochen**	
Aufwand	—	**22** PT	

Zweiundzwanzig Personentage und neun Wochen Umstellung für vier Zeitfelder — der teuerste Fall dieses Bandes. Er ist es wert, weil die vorletzte Zeile eine regulatorische Pflicht betrifft, die sich nachträglich nicht herstellen lässt.

Der Altbestand.

Umgang	Gewählt?	
Abnahme aus Schreibzeit ableiten	**nein**	**erfundene Daten**
Alte Werte löschen	nein	
Als unsicher kennzeichnen	**ja**	412 Mio. Werte
Stichtag nennen	ja	**ab da verlässlich**

Die erste Zeile war der naheliegende Vorschlag und wurde verworfen: Aus dem Schreibzeitpunkt eine Abnahmezeit zu berechnen erzeugt Werte, die richtig aussehen und es nicht sind. Vierhundertzwölf Millionen Befunde tragen jetzt ein Kennzeichen — unbequem und ehrlich.

Der Fehlschluss

Naheliegend	Richtig?	
"Immer alle Zeitpunkte speichern"	**nein**	was gebraucht wird
"Der Schreibzeitpunkt ist wertlos"	**nein**	**er bleibt**
"96 % richtig ist gut genug"	nein	**die 4 % sind sortiert**
"Ein Feld ist keine Bedeutung"	**ja**	

Die zweite Zeile ist wichtig: Der Schreibzeitpunkt wird nicht ersetzt, sondern ergänzt — er ist die Systemzeit und beantwortet die Nachvollzugsfrage. Falsch war nicht das Feld, sondern dass es als fachliche Zeit gelesen wurde.

Die dritte Zeile ist die statistische Falle: Die vier Prozent falsch sortierter Verläufe sind nicht zufällig verteilt — sie häufen sich bei Mikrobiologie und Fremdbefunden, also genau dort, wo eine Nacherfassung stattfindet und ein Verlauf klinisch etwas bedeutet.

Kleppmann[18] unterscheidet die beiden Zeitachsen und ihre Verwendung; Evans[9] begründet, warum ein Feldname eine fachliche Aussage trägt.

Fall 13

Die Wiederholung ohne Kennung

Die Vorlage

Zusammenhang. Übertragung, Go. Sendet Messwertblöcke an die Speicherung und wiederholt bei Fehlschlag.

```
func sende(ctx context.Context, b Block) error {
var letzter error
for versuch := 0; versuch < 5; versuch++ {
if err := ziel.Schreibe(ctx, b); err == nil {
return nil
} else {
letzter = err
}
warte(versuch)    // verdoppelnd, mit Streuung
}
return letzter
}
```

Zwölf Zeilen mit begrenzter Versuchszahl, verdoppelnden Abständen und Streuung — alles, was Band 23, Kapitel 10 verlangt. Die Wiederholung ist vorbildlich umgesetzt.

Die Diagnose

Was bei einem Zeitablauf geschieht.

Tatsächlich	Sender sieht	Folge
Nicht angekommen	Zeitablauf	Wiederholung richtig
Unterwegs	Zeitablauf	**doppelt**
Geschrieben, Antwort weg	**Zeitablauf**	**doppelt**

Drei Lagen, ein Signal — der Sender kann sie nicht unterscheiden. In zwei von drei Fällen erzeugt die Wiederholung einen doppelten Eintrag, und der Block ist nicht idempotent: Er enthält keine Kennung, an der die Speicherung eine Wiederholung erkennen könnte.

Was im Bestand gefunden wurde.

Befund	Anzahl	
Doppelte Messwerte	**31 400**	**in** 18 **Monaten**
Davon bemerkt	**0**	
Wirkung auf Verläufe	Punkte doppelt	unauffällig
Wirkung auf Mittelwerte	**verzerrt**	**Gleitmittel**

Einunddreißigtausend doppelte Werte in achtzehn Monaten, keiner davon bemerkt. Im Verlauf liegen zwei Punkte übereinander und fallen nicht auf — im Gleitmittel bekommt der doppelte Wert das doppelte Gewicht.

Das Urteil: überarbeitet

```
type Block struct {
Geraet  GeraetID
Folge   uint64    // fortlaufend je Geraet
Werte   []Messwert
}

// Speicherung:  EINFUEGEN ...  BEI KONFLIKT (geraet, folge)
//             NICHTS TUN
```

Zwei Felder im Block und eine Bedingung beim Einfügen. Die Kennung kommt vom Sender — Band 23, Kapitel 2 — und besteht aus Gerät und Folgenummer, die das Gerät ohnehin führt.

Die Begründung

	Vorlage	Überarbeitet	
Zeilen (Sender)	12	**12**	unverändert
Felder je Block	1	**3**	
Doppelte Werte	**31 400**	**0**	
Schreibdauer	8 ms	**9** ms	+12 %
Zusätzlicher Index	—	**1,4** GB	
Aufwand	—	**3** PT	

Die erste Zeile ist bemerkenswert: Am Sender ändert sich keine Zeile. Die Wiederholung war richtig — es fehlte die Voraussetzung, unter der sie zulässig ist. Die Behebung liegt vollständig im Vertrag und in der Speicherung.

Was mit den 31 400 **geschah.**

Vorgehen	Gewählt?	
Löschen	**nein**	**Nachvollzug**
Als Doppel markieren	**ja**	Auswertung ignoriert
Unverändert lassen	nein	Mittelwerte bleiben falsch

Gelöscht wird nichts — in einem Bestand mit Aufbewahrungspflicht ist das Markieren die einzige zulässige Form der Korrektur. Die Auswertung überspringt markierte Werte, die Historie bleibt vollständig.

Der Fehlschluss

Naheliegend	Richtig?	
“Nicht wiederholen”	**nein**	dann Verlust
“Der Empfänger soll erkennen”	**nein**	**ohne Kennung unmöglich**
“Zeitstempel als Kennung”	nein	**Uhr unzuverlässig**
“Wiederholung setzt Idempotenz voraus”	**ja**	

Die dritte Zeile war der erste Vorschlag und wäre in diesem Vorhaben besonders falsch gewesen: Das Erfassungsgerät hat keine verlässliche Uhr — Fall aus Band 23, Kapitel 4. Zwei Blöcke könnten denselben Zeitstempel tragen oder derselbe zwei verschiedene.

Die Folgenummer war die richtige Wahl, weil das Gerät sie ohnehin führte. Die beste Kennung ist meist keine neue, sondern eine, die es schon gibt und die man nur übertragen muss.

Waldo und Mitautoren[27] erklären, warum ein Zeitablauf drei Lagen nicht unterscheidet; Nygard[22] behandelt Wiederholung mit ihren Voraussetzungen.

Fall 14

Die Meldung ohne Zusammenhang

Die Vorlage

Zusammenhang. Übertragung, Go. Fehlerbehandlung entlang der Verarbeitungskette.

```
func verarbeite(b Block) error {
w, err := entpacke(b)
if err != nil { return err }
if err := pruefe(w); err != nil { return err }
if err := speichere(w); err != nil { return err }
return nil
}
```

Sieben Zeilen, jeder Fehler geprüft, keiner verschluckt. Nach den Maßstäben aus Band 22, Kapitel 2 ist das die korrekte Form — der Fehlerpfad ist ausgeschrieben und vollständig.

Die Diagnose

Was im Protokoll ankommt.

```
ERROR uebertragung:  unexpected EOF
```

Frage	Beantwortet?	
Welches Gerät?	**nein**	
Welcher Block?	nein	
Welcher Schritt?	**nein**	drei kommen infrage
Wie oft?	zählbar	
Was tun?	**nein**	

Drei blanke `return err` — und am Ende steht eine Meldung, die nichts sagt. Der Fehler wurde korrekt behandelt und sein Zusammenhang unterwegs verloren. Band 22, Kapitel 2: Die Kette muss gepflegt werden, sonst gibt es keine.

Was die Aufklärung kostete.

Schritt	Dauer	
Meldung gesichtet	5 min	
Code gelesen	40 min	drei Kandidaten
Zusatzausgaben eingebaut	**2** h	
Bereitgestellt, gewartet	**2 Tage**	**Fehler ist selten**
Ursache gefunden	20 min	

Zwei Tage Wartezeit, weil der Fehler etwa einmal täglich auftritt und die zusätzlichen Ausgaben erst nach einer Bereitstellung wirkten. Mit Zusammenhang in der Meldung wäre die Aufklärung in fünf Minuten erledigt gewesen.

Das Urteil: überarbeitet

```
func verarbeite(b Block) error {
w, err := entpacke(b)
if err != nil {
return fmt.Errorf("entpacken %s/%d:  %w",
b.Geraet, b.Folge, err)
}
if err := pruefe(w); err != nil {
return fmt.Errorf("pruefen %s/%d:  %w",
b.Geraet, b.Folge, err)
}
...
}
```

Ergebnis im Protokoll: `uebertragung: entpacken A17/48213: unexpected EOF`. Wo, für wen, welcher Schritt, welche Ursache — in einer Zeile. Das `%w` erhält den ursprünglichen Fehler, sodass `errors.Is` weiterhin greift.

Die Begründung

	Vorlage	Überarbeitet	
Zeilen	7	**19**	+171 %
Aufklärungsdauer	**2 Tage**	**≈ 5 min**	
Fehler je Woche	≈ 7	≈ 7	gleich
Nach Ursache filterbar	nein	**ja**	
Aufwand	—	**1** PT	

Zwölf zusätzliche Zeilen — fast dreimal so lang. Das ist der meistgenannte Vorwurf gegen die Fehlerbehandlung in Go, und er trifft zu. Die zweite Zeile ist die Gegenrechnung: zwei Tage gegen fünf Minuten, bei einem Fehler, der wöchentlich auftritt.

Was in die Meldung gehört.

Angabe	Aufnehmen?	
Schritt	**ja**	
Kennungen	**ja**	Gerät, Folge
Ursprünglicher Fehler	ja	`%w`
Messwerte	**nein**	**Band 19, Kap. 6**
Patienten-kennung	nein	Protokolle wandern

Die letzten beiden Zeilen sind die Grenze: Kennungen ja, Inhalte nein. Ein Protokoll wandert in Auswertungssysteme mit anderen Zugriffsrechten, und ein Kaliumwert in einer Fehlerzeile ist eine Offenlegung.

Der Fehlschluss

Naheliegend	Richtig?	
“Jeden Fehler anreichern”	**nein**	**Kette wird Roman**
“Hilfsfunktion dafür”	nein	**versteckt den Pfad**
“Ausnahmen wären besser”	**vertretbar**	**andere Abwägung**
“Zusammenhang hinzufügen, wo er neu ist”	**ja**	

Die erste Zeile ist die Übertreibung: Wer auf jeder Ebene anreichert, bekommt Meldungen mit acht Gliedern, in denen dieselbe Kennung fünfmal steht. Die Regel lautet: Zusammenhang hinzufügen, wo man etwas weiß, das die tiefere Ebene nicht wusste.

Die dritte Zeile ist die einzige Stelle in diesem Band, an der ich ein anderes Urteil ausdrücklich für vertretbar halte: Mit Ausnahmen und Ablaufverfolgung hätte man denselben Zusammenhang ohne zwölf Zeilen bekommen — und dafür die Nachteile aus Band 22, Kapitel 1 eingehandelt.

Kernighan und Pike[16] behandeln Meldungen als Teil des Handwerks; Hunt und Thomas[15] verbinden sie mit der Haltung zum eigenen Bestand.

Teil IV

Wann man es lässt

Fall 15

Der Code, der schon gut war

Die Vorlage

Zusammenhang. Auswertung, Python. Bestimmt die Anlässe für eine Warnung. Eine Funktion, 58 Zeilen.

```
def anlaesse(b:  Befund, vorher:  Befund | None) ->
set[Anlass]:
a:  set[Anlass] = set()
if b.art is ErgArt.NICHTAUSWERTBAR:
return {Anlass.UNGUELTIG}  # 6 Zeilen
...              # 9 Zeilen Bereichsprüfung
...              # 14 Zeilen kritische Grenzen
...              # 18 Zeilen Änderung zum Vorbefund
...              # 11 Zeilen Steigung
return a
```

Achtundfünfzig Zeilen in einer Funktion, fünf erkennbare Abschnitte. Die naheliegende Empfehlung lautet: zerlegen. Fünf Funktionen zu je zehn bis fünfzehn Zeilen, jede mit einem Namen — so steht es in jedem Buch über Lesbarkeit.

Die Diagnose

Geprüft	Befund	
Zuständigkeit	eine	**ohne “und” sagbar**
Argumente	2	gut
Zustand	keiner	rein
Prüffälle	**34**	2 ms
Verstehenszeit	**47** s	gut
Zweige	9	flach
Änderungen in 2 Jahren	**6**	je eine Stelle

Sieben Prüfungen, sieben gute Befunde. Die einzige Auffälligkeit ist die Zeilenzahl — und die ist nach Band 20, Kapitel 5 ausdrücklich kein Maßstab. Der Maßstab ist die Zuständigkeit, und die ist eine: "Liefert die Anlässe, aus denen dieser Befund eine Warnung auslöst."

Was eine Zerlegung ändern würde.

	Vorlage	Zerlegt	
Funktionen	1	6	
Zeilen	58	**74**	+28 %
Verstehenszeit	**47** s	**61** s	1,3×
Sprünge beim Lesen	**0**	**5**	
Namen zu finden	—	5	"pruefe2"

Die Zerlegung macht es schlechter — gemessen, nicht vermutet. Der Grund steht in der vorletzten Zeile: Die fünf Abschnitte sind Schritte eines Verfahrens, keine eigenen Zuständigkeiten. Wer sie herauslöst, muss beim Lesen fünfmal springen.

Das Urteil: so gelassen

Maßnahme	Gewählt?	
Zerlegen	**nein**	**macht es schlechter**
Umbenennen	nein	Namen sind gut
Kommentare ergänzen	nein	Code ist klar
Nichts	**ja**	

Der einzige Eingriff war eine Zeile in der Durchsichtsvorlage: ein Hinweis, dass die Länge geprüft und für richtig befunden wurde. Damit muss der nächste Prüfer die Frage nicht erneut stellen.

Die Begründung

Die Prüffrage	Kann ich in einem Satz ohne “und” sagen, was sie garantiert?
Die Antwort	**Ja — “die Anlässe zu diesem Befund”**
Die Folge	Sie hat eine Zuständigkeit und bleibt eine Funktion

Band 20, Kapitel 5 nennt die Zeilenzahl ausdrücklich als untauglichen Maßstab und die Zuständigkeit als tauglichen. Dieser Fall ist die Probe darauf: Eine Regel wird gegen den ersten Eindruck angewandt, und das Ergebnis ist Zurückhaltung.

Wann das Urteil kippen würde.

Wenn	Dann	
Ein Abschnitt anderswo gebraucht	zerlegen	**Wieder-verwendung**
Ein Abschnitt einzeln prüfbar sein muss	zerlegen	
Verstehenszeit steigt über ≈ 90 s	**prüfen**	
Zweige über ≈ 15	prüfen	
Nur die Zeilen wachsen	**nichts**	

Vier Bedingungen, unter denen das Urteil anders ausfiele — und keine davon ist heute erfüllt. Damit ist das Urteil überprüfbar und nicht endgültig: Wenn die Funktion auf hundert Zeilen wächst, wird erneut gemessen.

Der Fehlschluss

Naheliegend	Richtig?	
“Lange Funktionen sind in Ordnung”	**nein**	**diese ist es**
“Zerlegen schadet”	nein	
“Messen entscheidet immer”	fast	
“Der erste Eindruck ist eine Frage, keine Antwort”	**ja**	

Die erste Zeile ist die Verallgemeinerung, die dieser Fall gerade nicht stützt. Achtundfünfzig Zeilen waren hier richtig, weil sieben Prüfungen es zeigten — nicht weil Länge unerheblich wäre. Bei einer anderen Funktion derselben Länge kann das Urteil umgekehrt ausfallen.

Der eigentliche Ertrag ist die letzte Zeile: “Das sieht lang aus” ist ein Anlass hinzusehen, kein Befund. Wer aus dem Anlass sofort eine Maßnahme macht, hat die Prüfung übersprungen — und in diesem Fall hätte er es verschlechtert.

Pikes Merksätze[25] halten fest, dass man vor der Änderung misst; Kennedy[14] zeigt, wie man Verstehenszeit als Maßstab verwendet.

Fall 16

Die Optimierung, die bleibt

Die Vorlage

Zusammenhang. Messerfassung, C. Wandelt Rohwerte in Messwerte um. Läuft 500 mal je Sekunde, harte Schranke von 2 ms.

```
/* Umrechnung ueber Tabelle statt Division.
Gemessen 2024-03:  Division 41 ns, Tabelle 3 ns.
Bei 500/s und 2 ms Schranke relevant, siehe Messung.  */
static const int32_t kehrwert[257] = { ...  };

static inline int32_t umrechne(int32_t roh, uint8_t kanal)
{
return (int32_t)(((int64_t)roh * kehrwert[kanal]) » 16);
}
```

Eine Festkommamultiplikation mit Verschiebung statt einer Division, dazu eine Tabelle mit zweihundertsiebenundfünfzig vorberechneten Kehrwerten. Das ist genau die Sorte Code, die in jeder Durchsicht als verfrühte Optimierung angestrichen wird.

Die Diagnose

Prüffrage	Antwort	
Gemessen?	**ja**	**im Kommentar**
Auf der Zielhardware?	ja	nicht am Schreibtisch
Im heißen Pfad?	**ja**	500/s
Schranke betroffen?	**ja**	2 ms hart
Genauigkeit geprüft?	ja	41 Prüffälle
Verstehenszeit	**72** s	erhöht

Fünf von fünf Voraussetzungen erfüllt, und der Preis — zweiundsiebzig Sekunden Verstehenszeit gegen etwa zwanzig für eine Division — ist im Kommentar begründet. Das unterscheidet diesen Fall von einer verfrühten Optimierung.

Die Nachmessung auf heutiger Hardware.

Fassung	2024	2026	
Division	41 ns	**38** ns	
Tabelle	**3** ns	**3** ns	12,7×
Anteil am Zyklus	—	**1,9** %	
Ohne Tabelle	—	**2,8** %	

Die Nachmessung ist der eigentliche Beitrag dieses Falls — und sie schwächt die Begründung: Neunhundertstel eines Prozents des Zyklus. Ohne die Tabelle wäre die Schranke immer noch mit großem Abstand eingehalten.

Das Urteil: so gelassen

Erwägung	Gewicht	
Gewinn ist gering	gegen	0,9 %
Code ist schwerer	gegen	72 s
Läuft seit 2 Jahren	**dafür**	41 **Prüffälle grün**
Medizinprodukt	**dafür**	**Änderung = Nachweis**
Begründung dokumentiert	dafür	

Die vierte Zeile entscheidet: In einem zugelassenen Medizinprodukt zieht jede Änderung an der Messkette einen Nachweis nach sich — Prüfung, Dokumentation, Freigabe. Etwa acht Personentage, um neunhundertstel Prozent Laufzeit zurückzugeben und die Lesbarkeit zu verbessern.

Die Begründung

	Lassen	Ersetzen	
Aufwand	**0**	**8** PT	
Risiko	keins	Messkette	
Verstehenszeit	72 s	**21** s	3,4×
Zyklusanteil	1,9 %	2,8 %	
Wer liest den Code?	≈ 2 **mal im Jahr**		

Die letzte Zeile ist die entscheidende Größe und wird fast nie erhoben: Diese Funktion wird etwa zweimal im Jahr gelesen. Einundfünfzig Sekunden Ersparnis je Lesevorgang, zweimal im Jahr — gegen acht Personentage.

Was stattdessen geschah.

Maßnahme	Aufwand	
Nachmessung eingetragen	20 min	**Kommentar ergänzt**
Vermerk: entbehrlich	5 min	**bei nächster Änderung**
Code geändert	**0**	

Fünfundzwanzig Minuten statt acht Personentage. Der Kommentar trägt jetzt die Nachmessung von 2026 und den Hinweis, dass die Optimierung entbehrlich geworden ist — damit fällt sie beim nächsten ohnehin fälligen Eingriff.

Der Fehlschluss

Naheliegend	Richtig?	
"Optimierungen behält man"	**nein**	**diese wird fallen**
"Verfrühte Optimierung ist schlimm"	teils	**diese war begründet**
"Nachmessen lohnt nicht"	**nein**	es hat entschieden
"Der Aufwand des Rückbaus zählt mit"	**ja**	

> *Die erste Zeile ist die Verallgemeinerung, die dieser Fall nicht stützt. Die Optimierung bleibt nicht, weil sie gut ist, sondern weil ihr Rückbau in diesem regulatorischen Umfeld teurer ist als ihr Nachteil. In einem gewöhnlichen Dienst wäre sie längst gefallen.*
>
> *Knuths Satz über verfrühte Optimierung wird meist ohne seinen zweiten Teil zitiert: Er richtet sich gegen Optimierung ohne Messung, nicht gegen Optimierung. Dieser Code ist gemessen, dokumentiert und geprüft — er war nie das, wogegen der Satz sich richtet.*

Knuth[19] formulierte den Satz über verfrühte Optimierung — samt der Einschränkung, die meist fehlt; Bentley[2] zeigt, wie man den Gewinn misst, bevor man ihn behauptet.

Fall 17

Der doppelte Code, der bleibt

Die Vorlage

Zusammenhang. Zwei Prüffunktionen in verschiedenen Paketen, seit vier Jahren fast identisch.

```
# paket labor
def pruefe_bereich(wert, unten, oben):
if wert < unten:  return Lage.UNTER
if wert > oben: return Lage.UEBER
return Lage.INNERHALB

# paket abrechnung
def pruefe_spanne(betrag, min_, max_):
if betrag < min_:  return Spanne.UNTER
if betrag > max_: return Spanne.UEBER
return Spanne.INNERHALB
```

Zwei Funktionen, fünf Zeilen, Zeile für Zeile dieselbe Struktur. Ein Werkzeug zur Doppelungserkennung meldet sie zuverlässig, und in jeder Durchsicht kommt der Vorschlag, sie zusammenzulegen.

Die Diagnose

Die Prüffrage	Ändern sich beide aus demselben Grund?
Die Antwort	**Nein**

Änderungsgrund	Labor	Abrechnung
Neue Referenzlogik	**ja**	nein
Altersabhängige Grenzen	**ja**	nein
Steuerliche Rundung	nein	**ja**
Währungswechsel	nein	ja
Nachweisgrenzen	ja	nein

Fünf absehbare Änderungsgründe, kein einziger gemeinsamer. Die beiden Funktionen sehen gleich aus und haben nichts miteinander zu tun — Band 20, Kapitel 14: gleiches Aussehen ist kein gemeinsamer Grund.

Was in vier Jahren geschah.

	Änderungen	
Nur Labor	**3**	**Grenzwertarten**
Nur Abrechnung	**2**	
Beide gemeinsam	**0**	**in vier Jahren**

Fünf Änderungen, keine gemeinsame. Hätte man die Funktionen vor vier Jahren zusammengelegt, wären es fünf Änderungen an einer gemeinsamen Funktion gewesen — jede mit einem Schalter, der nur eine Seite betrifft.

Das Urteil: so gelassen

Maßnahme	Gewählt?	
Zusammenlegen	nein	**kein gemeinsamer Grund**
Gemeinsame Basisklasse	nein	schlimmer
Werkzeugmeldung abschalten	**ja**	**mit Begründung**
Code geändert	**0** Zeilen	

Die dritte Zeile ist der einzige Eingriff: Die Doppelungsmeldung wird für diese Stelle abgeschaltet, mit einem Verweis auf diesen Fall. Sonst kommt der Vorschlag bei jeder Durchsicht neu — und irgendwann setzt ihn jemand um.

Die Begründung

	Getrennt	Zusammen-gelegt	
Zeilen heute	**10**	**6**	kürzer
Zeilen nach 5 Änderungen	**19**	**31**	
Schalter	**0**	**5**	
Pakete gekoppelt	nein	**ja**	
Änderung trifft	1 Paket	2 **Pakete**	

Heute wären vier Zeilen gespart; nach fünf Änderungen sind es zwölf Zeilen mehr und fünf Schalter. Die zweite Zeile ist keine Vermutung — die fünf Änderungen sind bereits eingetreten, und man kann beide Verläufe nachrechnen.

Die schwerwiegendere Folge.

Bei Zusammen-legung	Folge	
Gemeinsames Paket nötig	**ja**	`util`?
Labor hängt an Abrechnung	mittelbar	
Änderung braucht beide Freigaben	**ja**	**zwei Bereiche**

Die dritte Zeile ist der eigentliche Preis und hat nichts mit Code zu tun: Eine gemeinsame Funktion bedeutet, dass jede Änderung an der Grenzwertlogik des Labors die Zustimmung der Abrechnung braucht. Vier Zeilen gegen eine organisatorische Kopplung.

Der Fehlschluss

Naheliegend	**Richtig?**	
"Doppelter Code ist in Ordnung"	**nein**	**dieser ist es**
"Dop-pelungswerkzeuge taugen nichts"	nein	**sie fragen**
"Fünf Zeilen sind zu wenig zum Teilen"	nein	Länge entscheidet nicht
"Gleiches Aussehen ist kein gemeinsamer Grund"	**ja**	

Die dritte Zeile ist die Falle: Bei fünf Zeilen wirkt die Doppelung harmlos und die Entscheidung beliebig. Sie ist es nicht — dieselbe Prüfung bei zweihundert doppelten Zeilen würde genauso ausgehen, wenn die Änderungsgründe verschieden sind.

Umgekehrt: Zwei Funktionen, die verschieden aussehen und sich aus demselben Grund ändern, gehören zusammengelegt — und die findet kein Werkzeug. Das Werkzeug sieht Zeichen, die Prüffrage betrifft Gründe.

Fowler[10] nennt die Bedingungen für das Zusammenlegen; Beck[1] behandelt die kleinen Entscheidungen, in denen sich solche Fragen stellen.

Fall 18

Die doppelte Prüfung

Die Vorlage

Zusammenhang. Naht zwischen Übertragung und Speicherung. Beide Seiten prüfen dasselbe.

Prüfung	Sender	Empfänger
Schema	ja	ja
Einheit passt zur Größe	ja	ja
Wert im möglichen Bereich	ja	ja
Zeitpunkt plausibel	ja	ja
Kennung eindeutig	kann nicht	ja

Vier von fünf Prüfungen laufen zweimal — einmal beim Sender, einmal beim Empfänger. Zusammen etwa hundertvierzig Zeilen, die dasselbe tun. Die Aufforderung, das zu bereinigen, kommt regelmäßig.

Die Diagnose

Was jede Seite leistet.

Sender	Verhindert, dass Unsinn das Haus verlässt — meldet früh, wo der Fehler entsteht
Empfänger	**Schützt den Bestand vor einem fehlerhaften oder fremden Sender**

Zwei verschiedene Zwecke, die zufällig dieselben Bedingungen prüfen. Wer eine Seite entfernt, entfernt einen Zweck — nicht eine Doppelung.

Was in drei Jahren tatsächlich eintrat.

Ereignis	Gefangen von	
Fehler in der Umrechnung	**Sender**	früh gemeldet
Gerät mit neuer Betriebsfassung	**Empfänger**	**andere Einheit**
Fremdhaus, falsches Schema	**Empfänger**	
Prüfstand versehentlich verbunden	**Empfänger**	Testdaten
Eigener Fehler nach Umbau	**Sender**	

Fünf Ereignisse, drei davon nur vom Empfänger gefangen — und alle drei betreffen Sender, die man nicht kontrolliert. Die zweite Zeile ist der Fall aus Band 23, Kapitel 3: Ein Gerät bekam eine neue Betriebsfassung und sendete plötzlich in mg/dl.

Das Urteil: so gelassen

Maßnahme	Gewählt?	
Senderprüfung entfernen	**nein**	späte Meldung
Empfängerprüfung entfernen	**nein**	**Vertrauensgrenze**
Gemeinsame Bibliothek	**nein**	**koppelt beide**
Nichts	**ja**	

Die dritte Zeile ist der Vorschlag, der am vernünftigsten klingt und am schlechtesten ist: Eine gemeinsame Prüfbibliothek koppelt Sender und Empfänger an dieselbe Umsetzung. Ein Fehler darin wird von beiden Seiten gemacht — und die Prüfung des Empfängers hätte ihn gerade fangen sollen.

Die Begründung

	Beide	Nur Empfänger	
Zeilen	**140**	**70**	
Fehler früh gemeldet	ja	**nein**	
Aufwand Aufklärung	≈ 20 min	≈ 3 h	
Gefangene Ereignisse	**5**	3	
Laufzeit je Block	**0,9** ms	0,5 ms	unerheblich

Siebzig gesparte Zeilen gegen zwei ungefangene Ereignisse und drei Stunden statt zwanzig Minuten Aufklärung je Vorfall. Der Grund für den Unterschied: Der Sender kennt den Zusammenhang, in dem der Wert entstand — der Empfänger sieht nur eine Zahl.

Die regulatorische Seite.

Anforderung	Erfüllt durch	
Plausibilität am Eingang	**Empfänger**	zwingend
Fehler an der Quelle	Sender	gefordert
Unabhängige Prüfung	**beide**	**ausdrücklich**

Die dritte Zeile macht die Entscheidung leicht: Bei Medizinprodukten ist die unabhängige Prüfung an der Vertrauensgrenze keine Empfehlung. Eine gemeinsame Bibliothek würde die Unabhängigkeit aufheben und wäre begründungspflichtig.

Der Fehlschluss

Naheliegend	Richtig?	
“Doppelte Prüfung ist immer gut”	**nein**	**innerhalb sinnlos**
“Nie eine gemeinsame Bibliothek”	nein	**innerhalb ja**
“Der Sender ist überflüssig”	nein	
“An der Vertrauensgrenze prüfen beide”	**ja**	

Die erste Zeile ist die Übertreibung: Dieselbe Prüfung an drei Stellen innerhalb eines Programms ist Verschwendung — dort gibt es keine Vertrauensgrenze. Der Fall trägt nur, weil zwischen Sender und Empfänger eine liegt.

Die Prüffrage lautet deshalb: Liegt zwischen den beiden Stellen eine Grenze, an der die eine Seite der anderen nicht vertrauen kann? Innerhalb eines Prozesses lautet die Antwort nein, über eine Naht fast immer ja.

McConnell[21] sammelt die Belege zu defensiver Programmierung und ihren Grenzen; Ramalho[26] zeigt, wo Prüfungen innerhalb eines Programms entbehrlich sind.

Fall 19

Der Dienst, der bleibt

Die Vorlage

Zusammenhang. Der Benachrichtigungsdienst — ein eigener Prozess, der Warnungen an Station, Piepser und Mobilgerät zustellt.

Größe	Wert	
Zeilen	**1900**	klein
Aufrufer	**1**	nur die Auswertung
Eigene Datenhaltung	**nein**	
Gemeinsame Bereitstellung	**meist**	11 von 14 mal
Naht	1	≈ 18 PT Einstand

Fünf Anzeichen für einen überflüssigen Dienst: klein, ein einziger Aufrufer, keine eigenen Daten, fast immer gemeinsam bereitgestellt. Nach Band 23, Kapitel 14 ist die Prüffrage klar zu beantworten — und der Vorschlag lautet: zusammenlegen.

Die Diagnose

Die vier Gründe aus Band 23 im Einzelnen.

Grund	Erfüllt?	
Eigene Sprache	nein	**beide Python**
Eigenes Team	nein	
Eigenständig erneuerbar	**selten genutzt**	3 von 14
Andere Betriebsanforderung	**ja**	**siehe unten**

Drei von vier Gründen tragen nicht. Der vierte trägt — und er allein genügt, weil er sich nicht durch Sorgfalt ersetzen lässt.

Die Betriebsanforderung.

	Auswertung	Benach-richtigung
Verfügbarkeit	99,5 %	**99,99 %**
Wartungs-fenster	nachts	**keines**
Speicherbedarf	**6** GB	**180** MB
Neustart dauert	**40** s	**1,2** s
Darf ausfallen	**ja**	**nein**

Die letzte Zeile ist die klinische Anforderung: Eine kritische Warnung muss zugestellt werden, auch wenn die Verlaufsansicht gerade nicht verfügbar ist. Zwei Größenordnungen Unterschied in der geforderten Verfügbarkeit.

Das Urteil: so gelassen

Bei Zusammen-legung	Folge	
Verfügbarkeit	99,5 % **für beide**	44 h/Jahr
Neustart	40 s ohne Warnungen	
Speicher	6 GB je Instanz	teurer
Nachtwartung	**Warnungen aus**	**unzulässig**

Vierundvierzig Stunden im Jahr ohne Warnungszustellung — das ist die Folge der Zusammenlegung, und sie ist klinisch nicht vertretbar. Die Naht kostet achtzehn Personentage und kauft eine Verfügbarkeit, die anders nicht zu haben ist.

Die Begründung

	Getrennt	Zusammen	
Bestandteile	2	**1**	
Nähte	1	**0**	
Aufwand gespart	—	≈ 8 PT	
Warnungen verfügbar	**99,99 %**	**99,5 %**	
Ausfall je Jahr	**53** min	**44 h**	50×

Acht Personentage Ersparnis gegen Faktor fünfzig bei der Ausfallzeit für kritische Warnungen. Das ist keine knappe Abwägung — und sie beruht auf einer Größe, die in keiner Codedurchsicht sichtbar wird.

Warum die anderen Anzeichen täuschen.

Anzeichen	Erklärung	
Nur ein Aufrufer	**Zweck, nicht Größe**	
Klein	Zustellung ist einfach	
Meist gemeinsam bereitgestellt	**Bequem-lichkeit**	3 **mal getrennt**
Keine eigenen Daten	muss er nicht	

Die dritte Zeile ist die aufschlussreichste: Elf von vierzehn gemeinsamen Bereitstellungen sind Bequemlichkeit — und die drei getrennten waren genau die Fälle, in denen es darauf ankam. Die Häufigkeit misst die Gewohnheit, nicht die Notwendigkeit.

Der Fehlschluss

Naheliegend	Richtig?	
“Kleine Dienste sind in Ordnung”	**nein**	**dieser ist es**
“Ein Grund genügt immer”	nein	**dieser genügt**
“Verfügbarkeit rechtfertigt jede Naht”	**nein**	nur wo gefordert
“Betriebsanforderungen stehen nicht im Code”	**ja**	

Die letzte Zeile ist der Ertrag dieses Falls und gilt weit über ihn hinaus: Fünf von fünf Anzeichen sprachen für eine Zusammenlegung, und alle fünf waren im Code sichtbar. Die Größe, die dagegen sprach, war es nicht.

Daraus folgt eine Verfahrensregel: Bevor man zwei Bestandteile zusammenlegt, fragt man den Betrieb nach ihren Verfügbarkeitszielen. Fünf Minuten, die in diesem Fall achtzehn Personentage Fehlentscheidung verhindert haben.

Conway[6] erklärt, warum Schnitte den Zuständigkeiten folgen; Waldo und Mitautoren[27] erinnern daran, dass eine Naht Eigenschaften hat, die im Programm nicht vorkommen.

Fall 20

Das Werkzeug, das bleibt

Die Vorlage

Zusammenhang. Kein Code — ein Vorschlag. Die Auswertung soll von der bestehenden Prüf- und Bauumgebung auf ein modernes Rahmenwerk umgestellt werden.

Versprechen	Glaubhaft?	
Schnellere Bauläufe	**ja**	210 s → 40 s
Bessere Fehlermeldungen	ja	
Weniger Konfiguration	**ja**	340 → 60 Zeilen
Verbreiteter	ja	**leichtere Einstellung**

Vier Versprechen, alle zutreffend, alle nachprüfbar. Der Vorschlag ist gut begründet und kommt von jemandem, der die neue Umgebung kennt. Es gibt kein technisches Gegenargument.

Die Diagnose

Was die Umstellung tatsächlich umfasst.

Posten	Aufwand	
Umstellung selbst	**6** PT	wie veranschlagt
Bauläufe anpassen	**4** PT	
Prüfstrecke	**9** PT	nicht veranschlagt
Vier weitere Bestandteile	**22 PT**	**sonst zwei Welten**
Einarbeitung, 6 Personen	**12** PT	
Nachweis-dokumentation	**8** PT	Medizinprodukt
Summe	**≈ 61 PT**	**veranschlagt:** 6

Einundsechzig Personentage statt sechs. Der größte Posten ist die vierte Zeile und wird regelmäßig übersehen: Ein Werkzeug in einem von fünf Bestandteilen bedeutet zwei Bauwelten, zwei Anleitungen und zwei Wege, dieselbe Frage zu beantworten.

Was der Gewinn wert ist.

Ersparnis	Je Jahr	
Bauzeit, 170 s je Lauf	**≈ 9 PT**	1200 **Läufe**
Konfigurations-pflege	≈ 2 PT	
Fehlersuche	≈ 1 PT	
Summe	**≈ 12 PT/Jahr**	

Zwölf Personentage im Jahr gegen einundsechzig einmalig — die Umstellung amortisiert sich nach etwa fünf Jahren. Das ist keine Ablehnung, sondern eine Zahl, die niemand erhoben hatte.

Das Urteil: so gelassen

Erwägung	Gewicht	
Amortisation 5 Jahre	gegen	**länger als geplant**
Zwei Bauwelten	**gegen**	oder 22 PT mehr
Bestehendes läuft	gegen	keine Beschwerden
Bauzeit stört	dafür	210 s

Stattdessen wurde das Naheliegende gemessen: Von den zweihundertzehn Sekunden Bauzeit entfielen hundertdreißig auf einen ungünstig gesetzten Prüfschritt. Vier Stunden Arbeit brachten die Bauzeit auf achtzig Sekunden — die Hälfte des versprochenen Gewinns für ein Sechzigstel des Aufwands.

Die Begründung

	Umstellen	Messen
Aufwand	**61** PT	**0,5** PT
Bauzeit nachher	**40** s	**80** s
Gewinn erreicht	100 %	**76** %
Zwei Bauwelten	ja	nein
Rückweg möglich	schwer	**trivial**

Sechsundsiebzig Prozent des Gewinns für ein Hundertzwanzigstel des Aufwands. Der Rest — vierzig Sekunden Unterschied je Lauf — rechtfertigt keine Umstellung, die fünf Jahre braucht, um sich zu tragen.

Was aufgeschrieben wurde.

Der Vorschlag	mit vollständiger Aufwandsschätzung
Die Bedingungen	**Wann er wieder zu prüfen ist**
Das Datum	**in zwei Jahren, oder früher bei Anlass**

Ein abgelehnter Vorschlag ohne Wiedervorlage kommt in einem Jahr unverändert zurück, und dieselbe Diskussion wird noch einmal geführt. Mit Datum und Bedingungen wird aus der Ablehnung eine Vertagung — und die Zahlen liegen beim nächsten Mal schon vor.

Der Fehlschluss

Naheliegend	Richtig?	
“Neue Werkzeuge lohnen nie”	**nein**	**oft doch**
“Das Bestehende ist gut”	nein	es ist bezahlt
“Der Vorschlag war falsch”	**nein**	**er war unvollständig**
“Der Umstellungsaufwand gehört in die Rechnung”	**ja**	

Die dritte Zeile ist mir wichtig: Der Vorschlag war sachlich richtig in allem, was er behauptete. Er hat nur die Hälfte der Rechnung aufgemacht — den Nutzen vollständig, die Kosten teilweise. Das ist der Normalfall bei Werkzeugvorschlägen, und zwar auf beiden Seiten.

Die zweite Zeile ist der Fehlschluss in die Gegenrichtung, gegen den sich dieser Band ebenso wendet: Bestehendes ist nicht deshalb richtig, weil es besteht. Hätte die Messung nichts gebracht, wäre das Urteil anders ausgefallen.

Brooks[3] zeigt, warum kein Werkzeug die wesentliche Komplexität beseitigt; Hunt und Thomas[15] raten, Werkzeuge zu beherrschen statt sie zu wechseln.

Teil V

Anhang

Fall A

Die zwanzig Fälle

Übersicht

Nr.	Fall	Urteil	Die Prüffrage
1	Wert ohne Einheit	**überarbeitet**	Mehr als eine Darstellung möglich?
2	Acht Argumente	überarbeitet	Wandern Argumente gemeinsam?
3	Verfrühte Abstraktion	**umgebaut**	Ist der dritte Fall gekommen?
4	Feld, das fehlen darf	überarbeitet	Bedeutet Fehlen mehr als eines?
5	Schleife, die fragt	überarbeitet	Aufruf über eine Naht?
6	Prüfung verschwindet	**umgebaut**	Wie geprüft, wie ausgeliefert?
7	Puffer ohne Grenze	überarbeitet	Steht die Größe in der Signatur?
8	Goroutine ohne Ende	überarbeitet	Wie endet sie?
9	Kanal, der puffert	**umgebaut**	Verlust oder Verzögerung?
10	Hierarchie	**umgebaut**	Wie viele Notlösungen?
11	Ohne Ablaufzeit	überarbeitet	Wie lange darf es veralten?
12	Falsche Zeit	**überarbeitet**	Welcher Zeitpunkt ist gemeint?
13	Ohne Kennung	überarbeitet	Ist es idempotent?
14	Meldung ohne Bezug	überarbeitet	Weiß ich etwas, das tiefer fehlt?
15	Code, schon gut	**so gelassen**	Eine Zuständigkeit?
16	Optimierung	**so gelassen**	Was kostet der Rückbau?
17	Doppelter Code	**so gelassen**	Derselbe Änderungsgrund?
18	Doppelte Prüfung	**so gelassen**	Liegt eine Grenze dazwischen?
19	Dienst bleibt	**so gelassen**	Andere Betriebsanforderung?
20	Werkzeug bleibt	**so gelassen**	Ist der Umstellungsaufwand drin?

Die letzte Spalte ist der eigentliche Ertrag dieses Bandes. Zwanzig Fragen, die man an eine Vorlage stellt — keine davon beantwortet sich aus dem Code allein, und keine davon ist eine Regel.

Die zwanzig Fehlschlüsse

1	Nicht: jeder Wert braucht einen eigenen Typ
2	Nicht: höchstens drei Argumente
3	Nicht: Abstraktion ist schlecht
4	Nicht: keine optionalen Felder
5	Nicht: keine geschachtelten Schleifen
6	Nicht: der Übersetzer ist schuld
7	Nicht: den Puffer größer machen
8	Nicht: immer gepufferte Kanäle
9	**Nicht: kein Verlust ist immer besser**
10	Nicht: Vererbung ist schlecht
11	Nicht: ein gemeinsamer Speicher wäre besser
12	Nicht: der Schreibzeitpunkt ist wertlos
13	Nicht: den Zeitstempel als Kennung nehmen
14	Nicht: jeden Fehler anreichern
15	**Nicht: lange Funktionen sind in Ordnung**
16	**Nicht: Optimierungen behält man**
17	**Nicht: doppelter Code ist in Ordnung**
18	**Nicht: doppelte Prüfung ist immer gut**
19	**Nicht: kleine Dienste sind in Ordnung**
20	**Nicht: das Bestehende ist gut**

Die sechs unteren sind die gefährlichsten, weil sie bequem sind: Wer aus Teil IV mitnimmt, dass man nichts ändern muss, hat das Gegenteil gelernt. In jedem der sechs Fälle war die Zurückhaltung das Ergebnis einer Prüfung — nicht ihr Ersatz.

Was die Fälle gekostet haben

Urteil	Fälle	Aufwand	Je Fall
Überarbeitet	9	**34,5** PT	3,8
Umgebaut	5	**56** PT	11,2
So gelassen	6	**2,6** PT	**0,4**
Summe	**20**	**≈ 93 PT**	

Die dritte Zeile ist die aufschlussreichste: Die sechs Fälle ohne Änderung haben zusammen zweieinhalb Personentage gekostet — für Messung, Begründung und einen Vermerk. Nicht zu handeln ist billig, aber nicht umsonst.

Der teuerste und der billigste Fall.

	Aufwand	Ertrag
Fall 12, Zeit	**22** PT	**Nachvollzug möglich**
Fall 8, Goroutine	**0,5** PT	1 **GB je Monat**

Faktor vierundvierzig zwischen dem teuersten und dem billigsten Eingriff. Wer die Reihenfolge nach diesem Verhältnis wählt, beginnt bei Fall 8 — und das ist die richtige Reihenfolge.

Fall B

Landkarte und Glossar

Woher die Regeln stammen

Fall	Regel geprüft	Band
1, 12	Daten vor Verfahren	**20, Kap. 2**
2, 15	Zuständigkeit, nicht Länge	**20, Kap. 5**
3, 17	Regel der drei	**20, Kap. 14**
4	Bedeutung des Fehlens	**23, Kap. 3**
5	Aufruf über die Naht	**23, Kap. 2**
6, 7	Undefiniertes Verhalten	**21, Kap. 7**
8, 9	Nebenläufigkeit	**22, Kap. 4–6**
10	Zusammensetzung	**22, Kap. 8**
11	Zwischenspeicher	**23, Kap. 9**
13	Idempotenz	**23, Kap. 2**
14	Fehlerkette	**22, Kap. 2**
16	Messen vor Optimieren	**21, Kap. 2**
18	Vertrauensgrenze	**19, Kap. 4**
19, 20	Wann teilen	**23, Kap. 14**

Keine der zwanzig Prüfungen brauchte eine neue Regel. Alle standen in den Bänden 19 bis 23 — was fehlte, war die Entscheidung, ob sie im jeweiligen Fall greifen.

Begriffe

Änderungsgrund	Warum sich ein Stück Code ändert — das Maß für Zusammenlegung
Fehlschluss	Naheliegende Verallgemeinerung, die aus einem Fall nicht folgt
Notlösung	Umgehung, die eine Struktur nicht vorsah
So gelassen	Urteil nach Prüfung, nicht Unterlassung
Überarbeitet	Derselbe Entwurf, besser ausgeführt
Umgebaut	Der Entwurf selbst war falsch
Verstehenszeit	Zeit bis zur richtigen Antwort auf fünf Fragen zum Code
Vorlage	Der Code, wie er vorliegt, mit seinem Zusammenhang

Fall C

Kommentierte Quellen

[1] Beck, K.: *Implementation Patterns*. Addison-Wesley 2007.

Behandelt die kleinen Entscheidungen — Benennung, Sichtbarkeit, Aufteilung — und begründet sie einzeln. Dünn, unterschätzt und für die Fälle 15 und 17 unmittelbar brauchbar.

[2] Bentley, J.: *Programming Pearls*. 2. Aufl., Addison-Wesley 1999.

Zeigt an durchgearbeiteten Beispielen, wie man den Gewinn misst, bevor man ihn behauptet. Die Form dieses Bandes — Fall, Diagnose, Begründung — ist bei Bentley abgeschaut.

[3] Brooks, F.: *No Silver Bullet*. IEEE Computer 20(4), 1987.

Trennt wesentliche von zufälliger Komplexität. *Für Fall 20 die Grundlage: Ein Werkzeug beseitigt zufällige Komplexität und lässt die wesentliche unberührt — weshalb der versprochene Gewinn regelmäßig überschätzt wird.*

[4] Bryant, R., O'Hallaron, D.: *Computer Systems: A Programmer's Perspective*. 3. Aufl., Pearson 2015.

Erklärt, was auf der Maschine tatsächlich geschieht — Keller, Speicherlayout, Überlauf. Für Fall 7 die Antwort auf die Frage, was ein Kellerüberlauf anrichtet.

[5] Chisnall, D.: *C Is Not a Low-Level Language*. ACM Queue 16(2), 2018.

Zeigt, wie weit das C-Modell von heutigen Maschinen entfernt ist. Für Fall 6 die Erklärung, warum der Übersetzer Freiheiten hat, mit denen niemand rechnet.

[6] Conway, M.: *How Do Committees Invent?* Datamation 14(4), 1968.

Fünf Seiten über den Zusammenhang von Organisation und Systemstruktur. Für Fall 19 die Erinnerung, dass ein Schnitt auch organisatorische Gründe haben darf — wenn man sie benennt.

[7] Dijkstra, E.: *Notes on Structured Programming*. 1970.

Mahnt früh, dass Allgemeinheit ohne Anlass die Einsicht verstellt. Für Fall 3 der älteste Beleg gegen die verfrühte Abstraktion.

[8] Donovan, A., Kernighan, B.: *The Go Programming Language*. Addison-Wesley 2015.

Das Standardwerk zu Go. *Führt die Nebenläufigkeitsmuster mit ihren Fallstricken vor — Fall 8 ist einer der dort beschriebenen, in freier Wildbahn angetroffen.*

[9] Evans, E.: *Domain-Driven Design*. Addison-Wesley 2003.

Begründet, warum Fachbegriffe ausdrücklich sein müssen und ein Feldname eine Aussage trägt. Für die Fälle

4 und 12 die Grundlage.

[10] Fowler, M.: *Refactoring*. 2. Aufl., Addison-Wesley 2018.

***Der Katalog der Umgestaltungsschritte** — und, wichtiger für diesen Band, der Bedingungen, unter denen sie angezeigt sind. Fall 17 wendet die Bedingung an und kommt zum Nein.*

[11] Gamma, E., u. a.: *Design Patterns*. Addison-Wesley 1994.

Liefert das Vokabular für Zielformen. Heute weniger als Rezeptbuch zu lesen denn als Wörterbuch — für Fall 10 genau in dieser Rolle verwendet.

[12] Goldberg, D.: *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys 23(1), 1991.

Der Aufsatz zum Thema. Für Fall 1 die Antwort auf die Frage, welche Information ein `float` verliert — und warum Betrag und Exponent sie behalten.

[13] Gregg, B.: *Systems Performance*. 2. Aufl., Addison-Wesley 2020.

Liefert das Vorgehen, mit dem man den teuren Posten findet, bevor man ihn behebt. Für die Fälle 5 und 11 — die Frist in Fall 11 ist nach diesem Verfahren gemessen, nicht gewählt.

[14] Hanson, D.: *C Interfaces and Implementations*. Addison-Wesley 1996.

Zeigt an vollständigen Beispielen, wie Schnittstelle und Umsetzung getrennt werden. Für Fall 15 der Beleg, dass Verstehenszeit ein brauchbarer Maßstab ist.

[15] Hunt, A., Thomas, D.: *The Pragmatic Programmer*. 20-Jahre-Ausgabe, Addison-Wesley 2019.

Behandelt die Haltung zum eigenen Bestand — und rät, Werkzeuge zu beherrschen statt sie zu wechseln. Für die Fälle 14 und 20.

[16] Kernighan, B., Pike, R.: *The Practice of Programming*. Addison-Wesley 1999.

***Das kürzeste gute Buch über das Handwerk.** Behandelt Fehlermeldungen, Fehlersuche und Stil als das, was sie sind: erlernbare Fertigkeiten. Für Fall 14.*

[17] Kernighan, B., Ritchie, D.: *The C Programming Language*. 2. Aufl., Prentice Hall 1988.

Beschreibt die Zeichenkettenfunktionen und ihre Grenzen — knapp und vollständig. Für Fall 7 die Quelle, in der schon steht, was dort falsch gemacht wurde.

[18] Kleppmann, M.: *Designing Data-Intensive Applications*. O'Reilly 2017.

***Ordnet Konsistenz, Zeit und Verzögerung in einen Zusammenhang.** Für die Fälle 4, 9, 11 und 12 — insbesondere die Unterscheidung der beiden Zeitachsen.*

[19] Knuth, D.: *Structured Programming with go to Statements*. ACM Computing Surveys 6(4), 1974.

Enthält den Satz über verfrühte Optimierung — mit der Einschränkung, die beim Zitieren regelmäßig entfällt. Für Fall 16 in seiner vollständigen Fassung gelesen.

[20] Lattner, C.: *What Every C Programmer Should Know About Undefined Behavior*. 2011.

Erklärt aus der Sicht des Übersetzerbauers, wie aus undefiniertem Verhalten entfernter Code wird. Für Fall 6 die maßgebliche Darstellung.

[21] McConnell, S.: *Code Complete*. 2. Aufl., Microsoft Press 2004.

Sammelt die empirischen Belege zu defensiver Programmierung und ihren Grenzen. Umfangreich; für Fall 18 sind die Kapitel zu Vertrauensgrenzen einschlägig.

[22] Nygard, M.: *Release It!* 2. Aufl., Pragmatic Bookshelf 2018.

Das Buch über Ausfall. *Behandelt Wiederholung mit ihren Voraussetzungen und zeigt, warum großzügige Puffer den Zusammenbruch nur verschieben. Für die Fälle 9 und 13.*

[23] Ousterhout, J.: *A Philosophy of Software Design*. 2. Aufl., Yaknyam Press 2021.

Fordert tiefe Schnittstellen und misst Abstraktionen daran, wie viel sie verbergen. Für die Fälle 2, 3 und 10 — eine Abstraktion, die nichts verbirgt, ist keine.

[24] Parnas, D.: *On the Criteria To Be Used in Decomposing Systems into Modules*. CACM 15(12), 1972.

Fünf Seiten, die den Modulbegriff begründet haben. *Das Kriterium — schneide nach Geheimnissen, nicht nach Arbeitsschritten — entscheidet Fall 2.*

[25] Pike, R.: *Notes on Programming in C* und *Go Proverbs*. 1989 und 2015.

Zwei kurze Merksatzsammlungen im Abstand von sechsundzwanzig Jahren, erstaunlich konsistent. "Messen, bevor man ändert" steht in beiden — für die Fälle 8 und 15.

[26] Ramalho, L.: *Fluent Python*. 2. Aufl., O'Reilly 2022.

Zeigt, was die Sprache selbst leistet und wo Prüfungen innerhalb eines Programms entbehrlich sind. Für Fall 18 die Gegenprobe zur defensiven Haltung.

[27] Waldo, J., u. a.: *A Note on Distributed Computing*. Sun Microsystems 1994.

Erklärt, warum ein Zeitablauf drei Lagen nicht unterscheidet. *Für Fall 13 die Begründung, warum Wiederholung ohne Idempotenz unzulässig ist — und für Fall 19, dass eine Naht Eigenschaften hat, die ein Aufruf nicht hat.*

[28] Wirth, N.: *Algorithms + Data Structures = Programs*. Prentice Hall 1976.

Der Titel ist die These und enthält eine Rangordnung: Die Datenstruktur kommt zuerst. Fall 1 ist eine Anwendung dieses Satzes auf eine Funktionssignatur.

Wenn Sie nur drei Bücher kaufen

[16]	**Kernighan, Pike**	**Für das Handwerk**
[23]	**Ousterhout**	Für das Urteil
[10]	**Fowler**	**Für die Schritte**

> *Kernighan und Pike lehren das Handwerk, Ousterhout die Maßstäbe, Fowler die Griffe. Auffällig ist, wie kurz die ersten beiden sind: zusammen etwa vierhundert Seiten für das, worauf es ankommt.*
>
> *Zwei Ergänzungen, beide frei verfügbar und beide unter zehn Seiten: [24] Parnas, das Kriterium für jeden Schnitt. Und [27] Waldo — acht Seiten gegen einen Irrtum, der in jedem Jahrzehnt zurückkehrt.*

Zwanzig Fälle, vierzehn Änderungen,
sechs Begründungen für Zurückhaltung.

— die Bilanz dieses Bandes
und der Reihe:
Die Regel sagt, wonach man fragt.
Der Fall sagt, was die Antwort war.